

同济大学计算机系

计算机组成原理课程综合实验报告



学 号 2351520

姓 名 毛星博

专 业 计算机科学与技术

授课老师 陈永生

一、 实验内容

本次实验旨在通过设计并实现一个支持 54 条 MIPS 指令的多周期 CPU，来深入理解计算机组成原理中数据通路、控制器、流水线以及异常处理等核心概念。实验将会对给出的 54 条 MIPS 指令进行功能分析、数据通路设计，设计并绘制出支持 54 条指令的完整多周期 CPU 数据通路图。根据指令流程图和操作时间表设计控制部件，使用 Verilog HDL 硬件描述语言，以模块化的方式完成各个功能部件及顶层 CPU 的设计，并对其进行功能验证（前仿真）和时序验证（后仿真）以及下板运行等测试。

二、 CPU 数据通路设计

(1) 首先，由于我们设计的是多周期的 CPU，需要先确定周期划分。

多周期 CPU 的 5 个基本状态（指令的执行流程被分解为多个状态，每个状态占用一个时钟周期）

状态 0: IF (Instruction Fetch - 取指)

$IR \leftarrow IMEM[PC]$ (从 PC 指向的指令存储器中读取指令，存入指令寄存器 IR)

$PC \leftarrow PC + 4$

状态 1: ID (Instruction Decode & Operand Fetch - 译码与取数)

译码 IR 中的指令。

$A \leftarrow RegFile[Rs]$ (读取 rs 寄存器的值到中间寄存器 A)

$B \leftarrow RegFile[Rt]$ (读取 rt 寄存器的值到中间寄存器 B)

$ACC \leftarrow PC + (SignExtend(IR[15:0]) \ll 2)$ (为分支指令提前计算跳转目标地址)

状态 2: EX (Execute - 执行)

根据指令类型执行 ALU 计算或地址计算。

状态 3: MEM (Memory Access - 访存)

使用 ACC 中的地址，读/写数据存储器 DMEM。

状态 4: WB (Write Back - 写回)

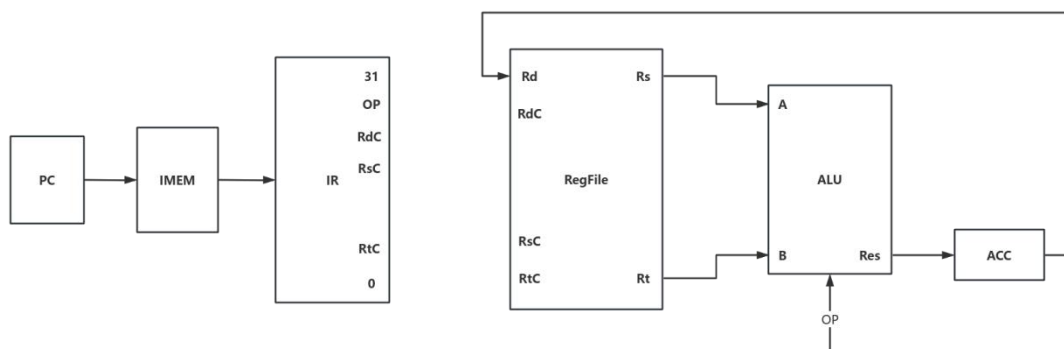
将结果写回通用寄存器堆。

(2) 根据每条指令所涉及部件和部件的数据输入来源，画出每条指令的数据通路，在已形成的数据通路下，画出每条指令从取指到执行过程的指令流程图。

R-Type

ADD/ADDU/SUB/SUBU/AND/OR/XOR/NOR/SLT/SLTU

这类指令使用来自两个源寄存器 (rs 和 rt) 的值进行计算

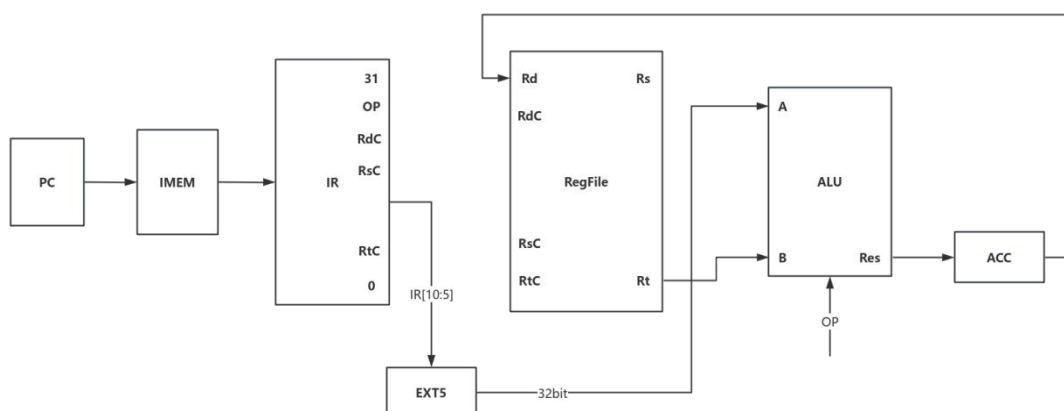


流程: IF -> ID -> EX -> WB (共 4 个周期)

- ◆ 周期 0 (IF): $IR \leftarrow IMEM[PC]$, $PC \leftarrow PC + 4$
- ◆ 周期 1 (ID): $A \leftarrow RegFile[Rs]$, $B \leftarrow RegFile[Rt]$
- ◆ 周期 2 (EX): $ACC \leftarrow A \text{ op } B$
- ◆ 周期 3 (WB): $RegFile[Rd] \leftarrow ACC$

SLL/SRL/SRA

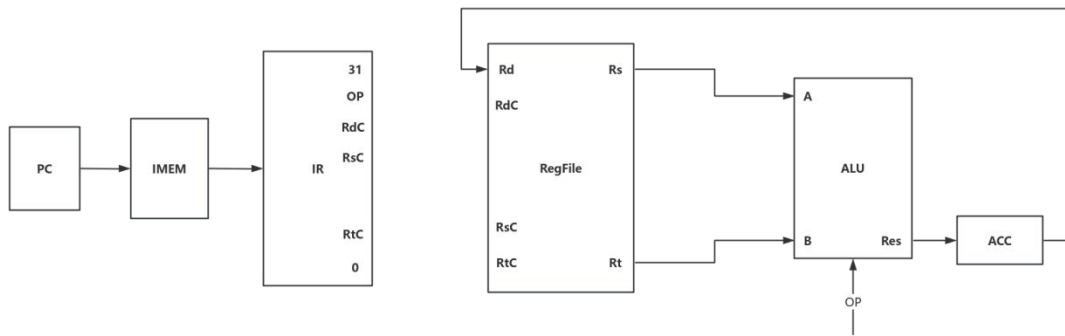
这类指令使用 rt 寄存器的值，并根据 rs 寄存器中的值进行移位



流程: IF -> ID -> EX -> WB (共 4 个周期)

- ◆ 周期 0 (IF): $IR \leftarrow IMEM[PC]$, $PC \leftarrow PC + 4$
- ◆ 周期 1 (ID): $B \leftarrow RegFile[Rt]$, $A \leftarrow IR[10:5]$
- ◆ 周期 2 (EX): $ACC \leftarrow B \ll A$
- ◆ 周期 3 (WB): $RegFile[Rd] \leftarrow ACC$

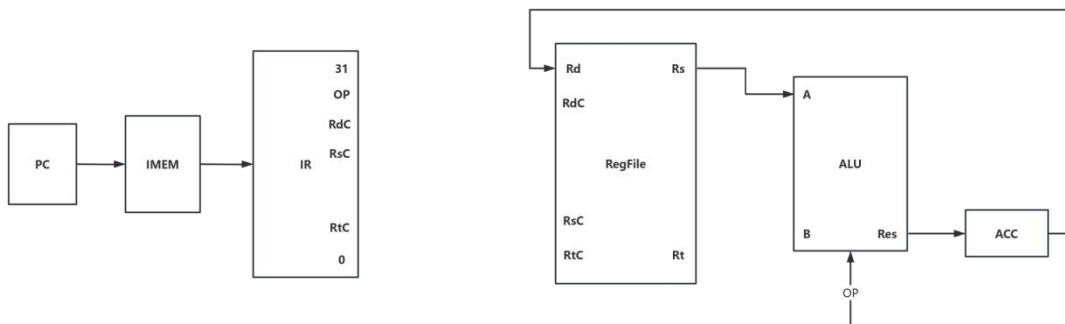
SLLV/SRLV/SRAV



流程: IF -> ID -> EX -> WB (共 4 个周期)

- ◆ 周期 0 (IF): $IR \leftarrow IMEM[PC]$, $PC \leftarrow PC + 4$
- ◆ 周期 1 (ID): $A \leftarrow RegFile[Rs]$, $B \leftarrow RegFile[Rt]$
- ◆ 周期 2 (EX): $ACC \leftarrow B \ll A$
- ◆ 周期 3 (WB): $RegFile[Rd] \leftarrow ACC$

CLZ 计算前导零个数

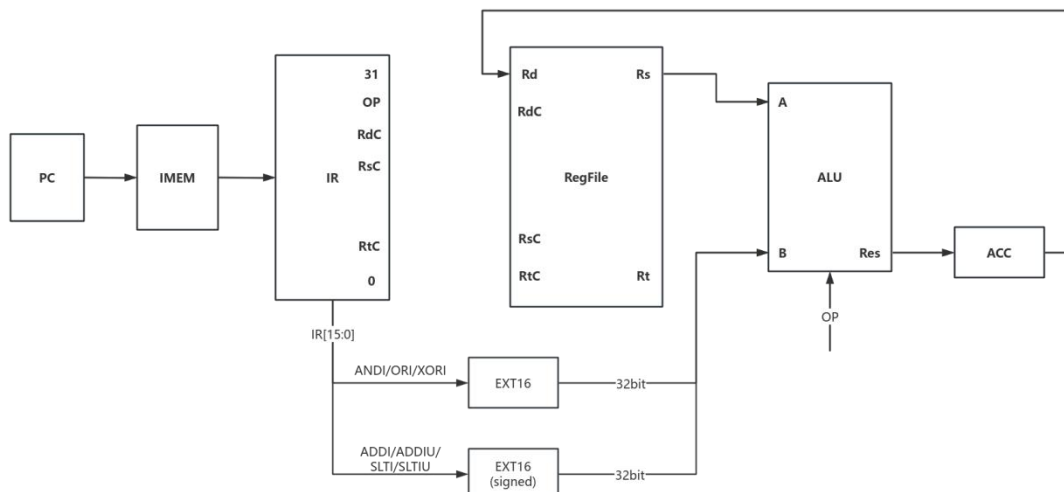


流程: IF -> ID -> EX -> WB (共 4 个周期)

- ◆ 周期 0 (IF): $IR \leftarrow IMEM[PC]$, $PC \leftarrow PC + 4$
- ◆ 周期 1 (ID): $A \leftarrow RegFile[Rs]$
- ◆ 周期 2 (EX): $ACC \leftarrow CountLeadingZeros(A)$
- ◆ 周期 3 (WB): $RegFile[Rd] \leftarrow ACC$

I-type

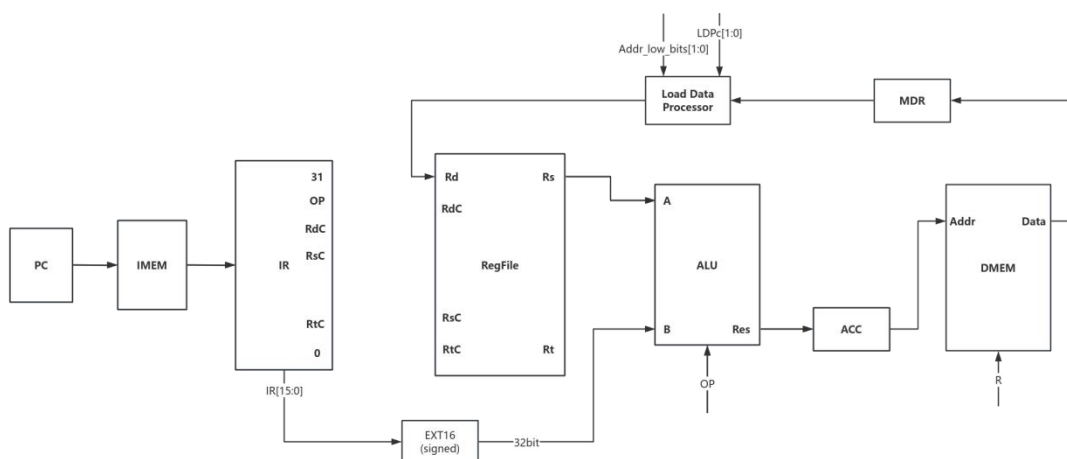
ADDI/ADDIU/ANDI/ORI/XORI/SLTI/SLTIU/LUI



流程: IF -> ID -> EX -> WB (共 4 个周期)

- ◆ 周期 0 (IF): $IR \leftarrow IMEM[PC]$, $PC \leftarrow PC + 4$
- ◆ 周期 1 (ID): $A \leftarrow RegFile[Rs]$
- ◆ 周期 2 (EX): $ACC \leftarrow A \text{ OP } SignExtend(IR[15:0]) / A \text{ OP } ZeroExtend(IR[15:0])$
- ◆ 周期 3 (WB): $RegFile[Rd] \leftarrow ACC$

LW/LB/LBU/LH/LHU

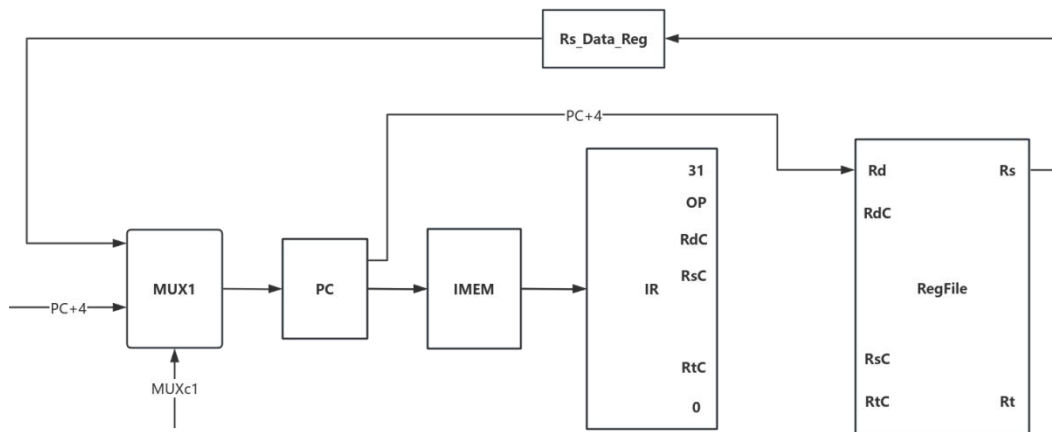


流程: IF -> ID -> EX -> MEM -> WB (共 5 个周期)

- ◆ 周期 0 (IF): $IR \leftarrow IMEM[PC]$, $PC \leftarrow PC + 4$
- ◆ 周期 1 (ID): $A \leftarrow RegFile[Rs]$
- ◆ 周期 2 (EX): $ACC \leftarrow A + SignExtend(IR[15:0])$
- ◆ 周期 3 (MEM): $MDR \leftarrow DMEM[ACC]$
- ◆ 周期 4 (WB): $RegFile[Rd] \leftarrow MDR$ (load data processor 处理)

SW/SH/SB

JR/JALR



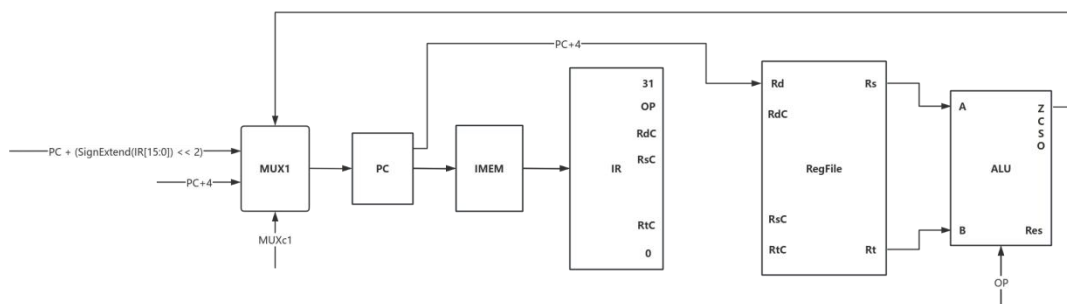
流程: IF -> ID -> EX (共 3 周期)

- ◆ 周期 0 (IF): $IR \leq IMEM[PC]$, $PC \leq PC + 4$.
- ◆ 周期 1 (ID): 译码。 $Rs_Reg \leq RegFile[Rs]$
- ◆ 周期 2 (EX): 从寄存器 Rs_Reg 中获取地址并更新 PC。

jr: $PC \leq Rs_Reg$.

jalr: $RegFile[Rd] \leq PC$ (将 PC+4 存入 Rd 指定的返回地址寄存器); $PC \leq Rs_Reg$.

BEQ/BNE

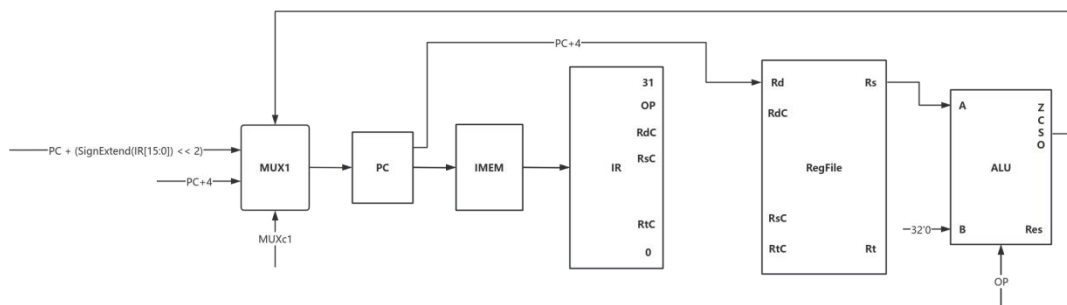


流程: IF -> ID -> EX (共 3 周期)

- ◆ 周期 0 (IF): $IR \leq IMEM[PC]$, $PC \leq PC + 4$.
- ◆ 周期 1 (ID): $A \leq RegFile[Rs]$, $B \leq RegFile[Rt]$
- ◆ 周期 2 (EX): 根据 Zero 标志位来判断条件是否成立

$PC \leq MUX1$

BGEZ

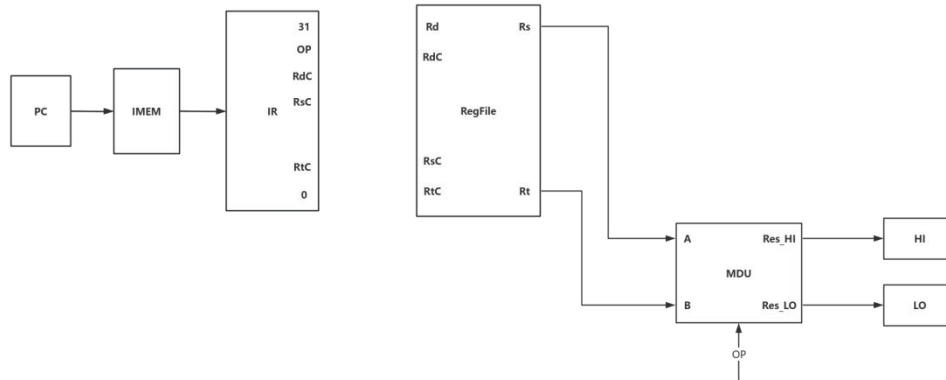


流程: IF -> ID -> EX (共 3 周期)

- ◆ 周期 0 (IF): $IR \leq IMEM[PC]$, $PC \leq PC + 4$.
 - ◆ 周期 1 (ID): $A \leq RegFile[Rs]$, $B \leq RegFile[Rt]$
 - ◆ 周期 2 (EX): 根据 negative 标志位来判断条件是否成立
- $PC \leq MUX1$

乘除法指令

MULT/MULTU/DIV/DIVU



流程: IF -> ID -> EX_Start -> ... -> EX_Done (多周期)

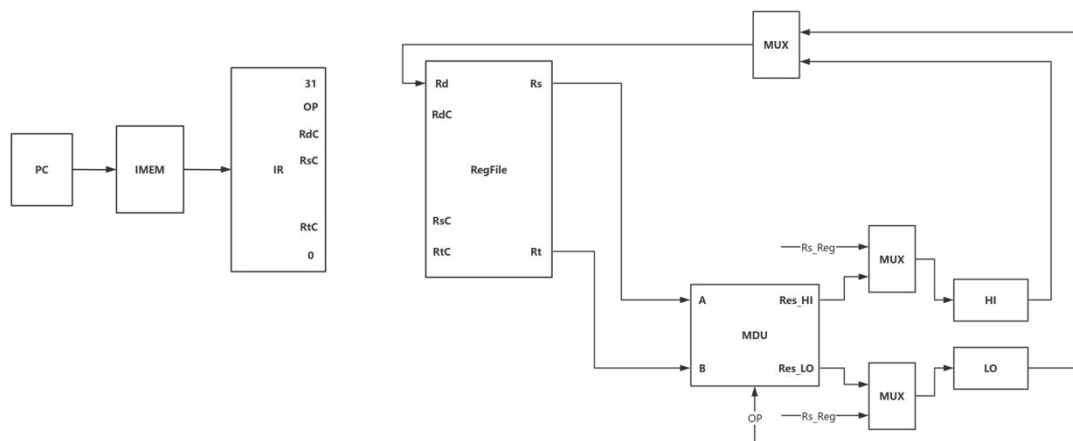
周期 1 (ID): $A \leq RegFile[Rs]$; $B \leq RegFile[Rt]$.

周期 2 (EX_Start): 启动乘法除法单元。

周期 3...N (EX_Wait - 循环等待)

周期 N+1 (EX_Done - 完成写入) $HI \leq Res_HI$, $LO \leq Res_LO$

MFHI/MFLO/MTHI/MTLO

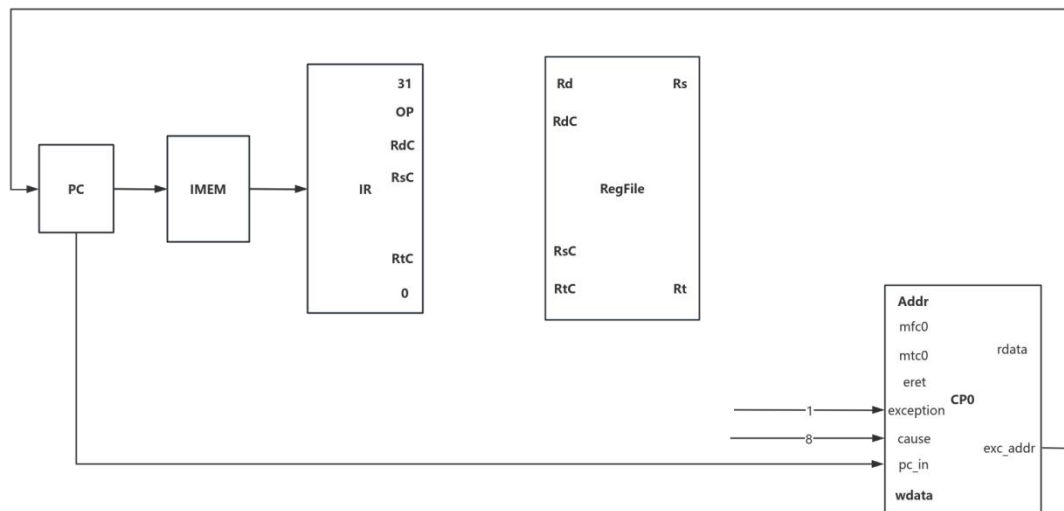


mfhi/mflo: IF -> ID -> WB (3 周期)。周期 2(WB): $RegFile[Rd] \leq HI$ (或 LO)

mthi/mtlo: IF -> ID -> EX (3 周期)。周期 1(ID): $Rs_Reg \leq RegFile[Rs]$ 。周期 2(EX): $HI \leq Rs_Reg$ (或 $LO \leq Rs_Reg$)

异常与系统调用

SYSCALL



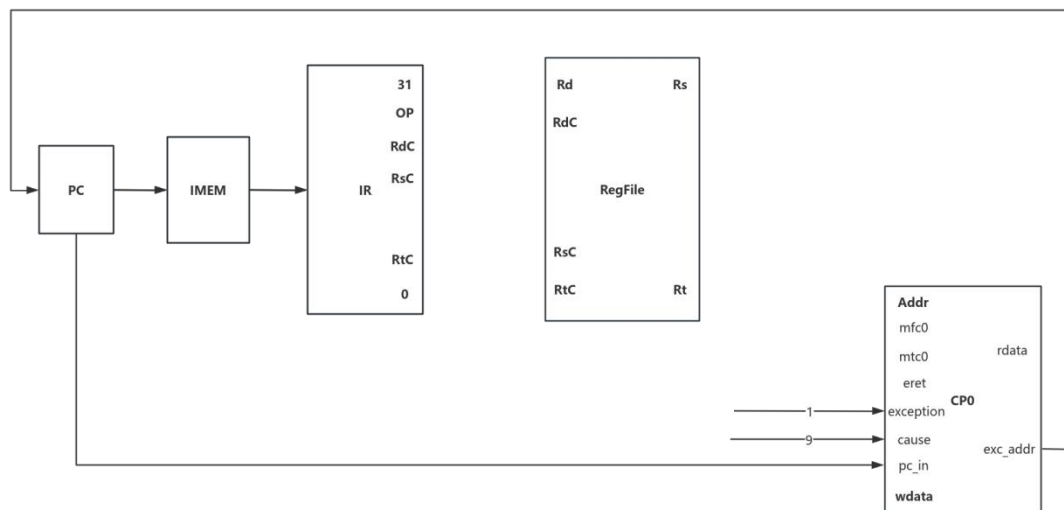
流程: IF -> ID -> EXCEPTION (3 周期)

周期 0 (IF - 取指) $IR \leq IMEM[PC]$, $PC \leq PC + 4$

周期 1 (ID - 译码)

周期 2 (EXCEPTION - 异常处理) $exception \leq 1$, $cause \leq 8$, $PC_i \leq PC - 4$, $PC \leq exc_addr$

BREAK



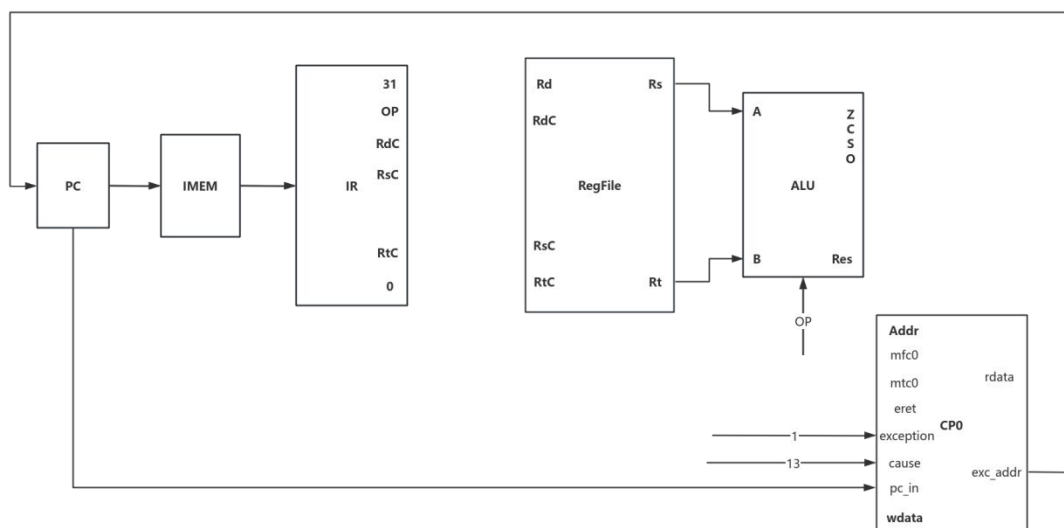
流程: IF -> ID -> EXCEPTION (3 周期)

周期 0 (IF - 取指) $IR \leq IMEM[PC]$, $PC \leq PC + 4$

周期 1 (ID - 译码)

周期 2 (EXCEPTION - 异常处理) $exception \leq 1$, $cause \leq 9$, $PC_i \leq PC - 4$, $PC \leq exc_addr$

TEQ



总流程 (若相等): IF -> ID -> EX -> EXCEPTION (共 4 个周期)

周期 0 (IF - 取指) $IR \leftarrow IMEM[PC]$, $PC \leftarrow PC + 4$

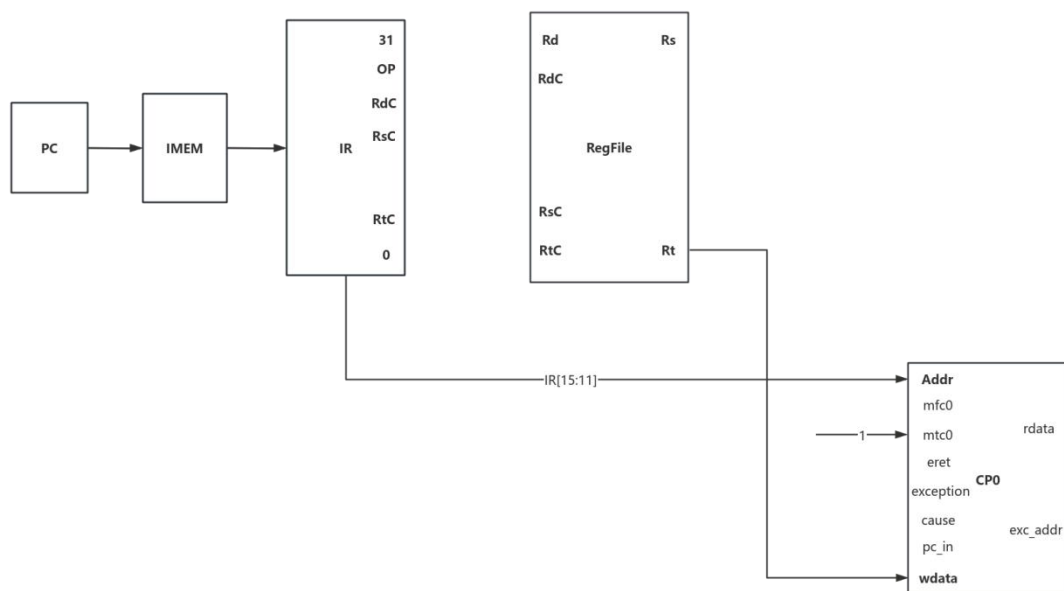
周期 1 (ID - 译码) $A \leftarrow RegFile[Rs]$, $B \leftarrow RegFile[Rt]$

周期 2 (EX - 执行/比较) $ACC \leftarrow A - B$

周期 3 (EXCEPTION - 异常处理) $Z = 1$

$exception \leq 1$, $cause \leq 13$, $PC_i \leftarrow PC - 4$, $PC \leftarrow exc_addr$

MTC0



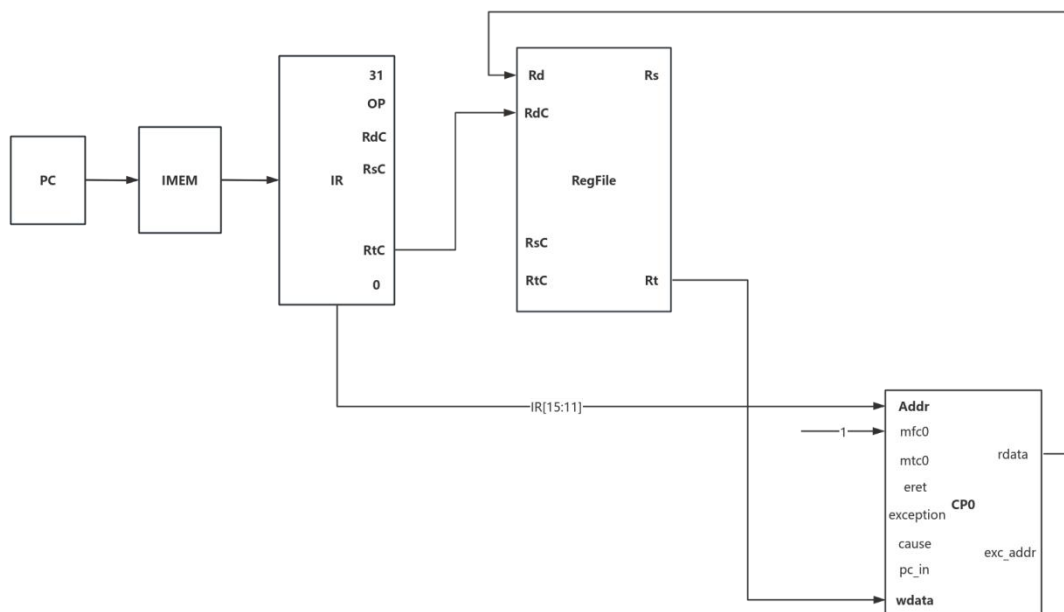
总流程: IF -> ID -> EX (共 3 个周期)

周期 0 (IF - 取指) $IR \leftarrow IMEM[PC]$, $PC \leftarrow PC + 4$

周期 1 (ID - 译码) $Rt_Reg \leftarrow RegFile[rt]$, $Addr \leftarrow IR[15:11]$

周期 2 (EX - 执行) $mtc0 \leftarrow 1$, $Addr \leftarrow IR[15:11]$, $wdata \leftarrow Rt_Reg$

MFC0



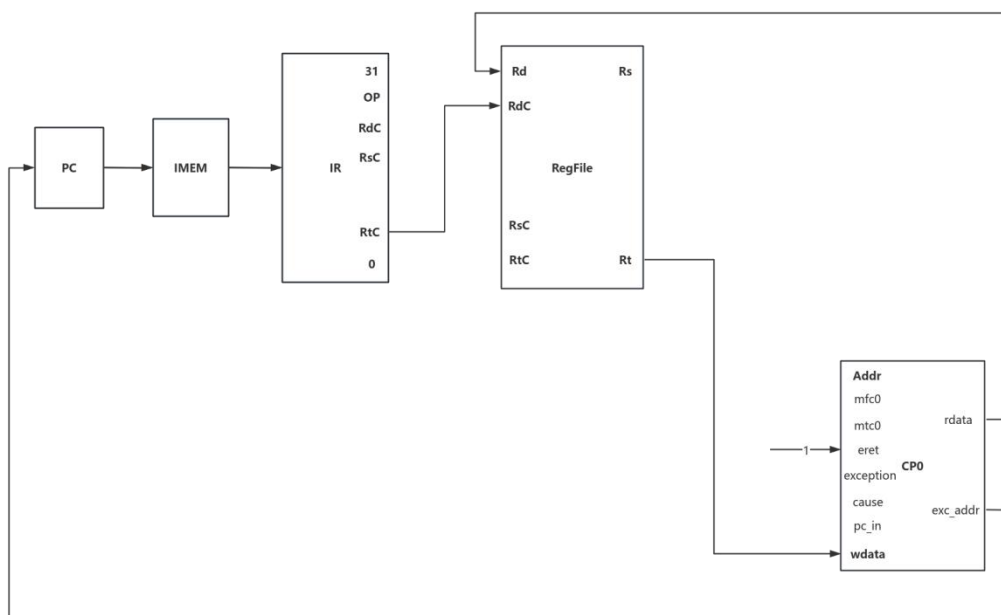
总流程: IF -> ID -> WB (共 3 个周期)

周期 0 (IF - 取指) $IR \leq IMEM[PC]$, $PC \leq PC + 4$

周期 1 (ID - 译码) $Addr \leq IR[15:11]$, $mfc0 \leq 1$

周期 2 (WB - 写回) $RegFile[Rt] \leq rdata$

ERET



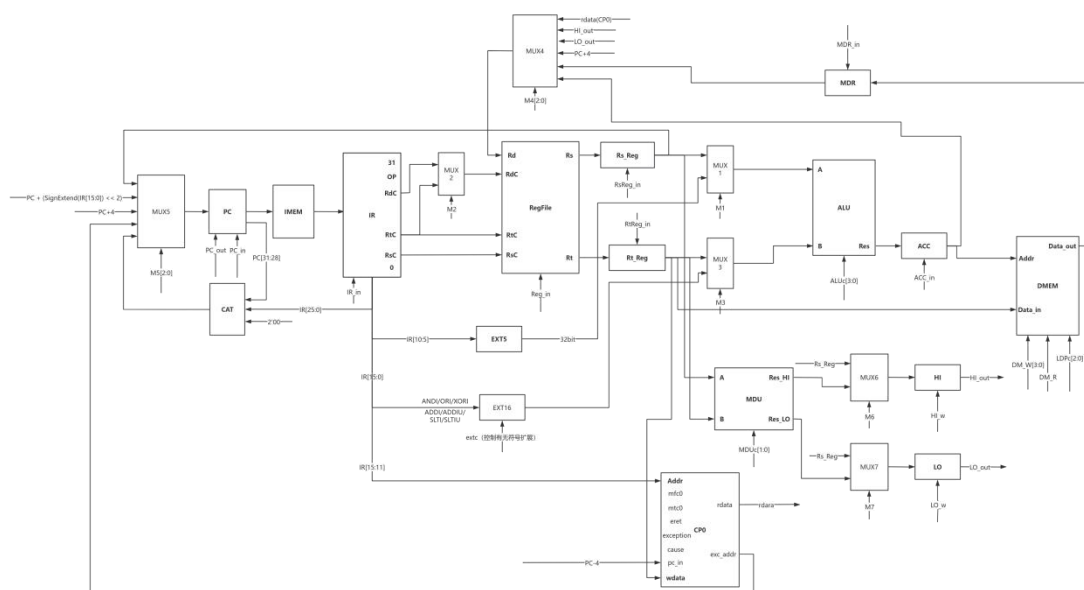
总流程: IF -> ID -> EX (共 3 个周期)

周期 0 (IF - 取指) $IR \leq IMEM[PC]$, $PC \leq PC + 4$

周期 1 (ID - 译码)

周期 2 (EX - 执行/状态恢复) $eret \leq 1$, $PC \leq exc_addr$

(3) 54 条 MIPS 指令总数据通路图



三、CPU 控制部件设计

(1) 确定数据通路中所有所需的控制信号

本 CPU 的控制器 (CU) 是一个有限状态机 (FSM)，其核心任务是根据当前所处的状态和指令的操作码，为数据通路上的所有部件生成一系列精确的控制信号。这些信号共同协作，引导数据在正确的通路中流动并被正确地处理。根据最终的数据通路设计，主控制器 (FSM) 需要生成以下控制信号：

1. 取指与 PC 控制信号

PC_in (PC 写使能)

说明: 1 位信号。当为 1 时，允许 PC 寄存器在时钟上升沿更新其值。

IR_in (指令寄存器写使能)

说明: 1 位信号。当为 1 时，允许指令寄存器 (IR) 锁存从指令存储器 (IMEM) 读出的新指令。仅在 IF 周期有效。

M5[2:0] (PC 来源选择)

说明: 3 位 Mux 选择信号，用于决定 PC 的下一个值来源。

3'b000: 选择 Rs_Reg 的输出 (用于 jr, jalr)。

3'b001: 选择分支地址计算单元的输出 (用于 beq, bne, bgez)。

3'b010: 选择 PC+4 加法器的输出 (用于顺序执行)。

3'b011: 选择来自 CP0 的 exc_addr 端口 (用于异常跳转和 eret 返回)。

3'b100: 选择 J-Type 跳转地址计算单元的输出 (用于 j, jal)。

2. 译码与寄存器堆 (RegFile) 控制

Reg_in (寄存器堆写使能)

说明: 1 位信号，即 RegWrite。当为 1 时，允许数据被写入寄存器堆。

M4[2:0] (写回数据选择)

说明: 3 位 Mux 选择信号，即 MemToReg，决定写回寄存器堆的数据来源。

3'b000: 来自 CP0 的 rdata 端口 (用于 mfc0)。
3'b001: 来自 HI 寄存器 (用于 mghi)。
3'b010: 来自 LO 寄存器 (用于 mfl0)。
3'b011: 来自 MUX4 内部锁存的 PC+4 值 (用于 jal, jalr)。
3'b100: 来自 LDP 模块的输出 (用于加载指令)。
3'b101: 来自 ACC 寄存器 (用于 ALU 运算指令)。

3. 执行单元 (ALU/MDU) 控制

ALUc[3:0]: 4 位 ALU 操作码, 规定 ALU 执行的具体运算。

M1 (ALUSrcA): 1 位 Mux 选择信号, 决定 ALU 的第一个操作数来源。

0: 来自 Rs_Reg。

1: 来自指令码中的 shamt 字段。

M3 (ALUSrcB): 1 位 Mux 选择信号, 决定 ALU 的第二个操作数来源。

0: 来自 Rt_Reg。

1: 来自立即数扩展器 Extender 的输出。

extc (扩展控制): 1 位信号, 当 M3=1 时有效, 规定立即数扩展方式。

0: 零扩展。

1: 符号扩展。

MDUc[1:0]: 2 位 MDU 操作码, 规定 MDU 执行的运算。

00: mult (有符号乘)

01: multu (无符号乘)

10: div (有符号除)

11: divu (无符号除)

MDU_start: 1 位信号, 用于在一个时钟周期内发出启动 MDU 运算的命令。

HI_w, LO_w: 1 位信号, 分别是 HI 和 LO 寄存器的写使能。

M6, M7: HI 和 LO 寄存器的输入源 Mux 选择信号。

0: 选择来自 Rs_Reg 的通路 (用于 mthi/mtlo)。

1: 选择来自 MDU 结果的通路 (用于 mult/div 等)。

4. 访存单元 (DMEM) 控制

DM_r: 1 位数据存储器读使能。

DM_w: 1 位数据存储器写使能。

byte_out_c[2:0]: 3 位加载数据处理控制信号, 即 LDPc。它告诉 LDP 模块当前执行的是哪种加载指令 (lw, lb, lbu 等), 以便进行正确的选择和扩展。

StoreOp[1:0]: (此信号替代您表格中的 byte_in_ena) 2 位存储类型控制信号。主控制器 FSM 仅输出该信号, 表示本次存储是字节、半字还是全字。一个专门的解码器会根据此信号和地址低位生成最终的 4 位 byte_in_ena 送给 DMEM。

5. CP0 与异常控制

mfc0, mtc0, eret, exception: 4 个独立的 1 位信号, 分别用于触发 CP0 的对应操作。

cause[4:0]: 5 位异常码。当 exception 为 1 时, 此信号向 CP0 提供具体的异常原因。

6. 流水线寄存器控制

(2) 根据指令流程图, 编排每条指令每个周期的操作时间表。

[illegible]

指令	sw					
周期	t1	t2	t3	t4	t4	t4
PC_in	1					
IR_in	1					
ACC_in			1			
ALUc[3:0]			ADDU (0000)			
MDUc[1:0]						
DM_r						
DM_w				1	1	1
byte_out_c[2:0]						
byte_in_ena[3:0]				1111(全字)	0011(低半字)	0001(低字节)
HL_w						
LO_w						
M1			0			
M2						
M3			1			
M4[2:0]						
M5[2:0]	010 (PC+4)					
M6						
M7						
mfc0						
mtc0						
eret						
exception						
cause[4:0]						
MDR_in						
Reg_in						
exc			1			
RsReg_in		1				
RtReg_in		1				
MDU_start						
Latch_PC_in_						
M4						

指令	j		jal		jr		jalr		beq		bne		bgez	
周期	t1	t2	t3	t2	t3	t1	t2	t3	t2	t3	t2	t3	t2	t3
PC_in	1		1		1	1		1		1		1		1
IR_in	1					1				1				
ACC_in														
ALUc[3:0]														
MDUc[1:0]														
DM_r														
DM_w														
byte_out_c[2:0]														
byte_in_ena[3:0]														
HL_w														
LO_w														
M1														
M2														
M3														
M4[2:0]														
M5[2:0]	010 (PC+4)		100 (CAT)		100 (CAT)	010 (PC+4)		000(Rs)		010 (PC+4)		001 (PC跳转)		001 (PC跳转)
M6														
M7														
mfc0														
mtc0														
eret														
exception														
cause[4:0]														
MDR_in														
Reg_in					1			1						
exc														
RsReg_in							1							
RtReg_in										1				
MDU_start														
Latch_PC_in_					1									
M4														

指令	mthi		mflr		mthi		mflr	
周期	t1	t2	t3	t3	t1	t2	t3	t3
PC_in	1				1			
IR_in	1				1			
ACC_in								
ALUc[3:0]								
MDUc[1:0]								
DM_r								
DM_w								
byte_out_c[2:0]								
byte_in_ena[3:0]								
HL_w							1	
LO_w								1
M1								
M2								
M3								
M4[2:0]			001 (Hi)	010 (Lo)				
M5[2:0]	010 (PC+4)				010 (PC+4)			
M6							0	0
M7							0	0
mfc0								
mtc0								
eret								
exception								
cause[4:0]								
MDR_in								
Reg_in			1	1				
exc								
RsReg_in						1		
RtReg_in								
MDU_start								
Latch_PC_in_								
M4								

指令	周期	syscal	break	irq	mfc0	mfc0	eret
PC_in	t1	t2	t3	t1	t2	t3	t1
IR_in	1		1	1			1
ACC_in							
ALU[3:0]				SUB(0011)			
MDU[1:0]							
DM.L							
DM.W							
byte_out_c[2:0]							
byte_in_ena[3:0]							
HI.W							
LO.W							
M1				0			
M2				0			
M3							
M4[2:0]						000(CPG-rdse)	
M5[2:0]	010 (PC+4)		011(exc_ad dr)	010 (PC+4)		011(exc_ad dr)	
M6							
M7							
mfc0							
eret						1	1
exception_cause[4:0]		1000	1001				
MDR_in							
Reg_in						1	
extc							
RsReg_in				1			
RtReg_in				1			
MDU_start					1		
Latch_PC_in							
M4							

指令	周期	mult	multu	div	divu
PC_in	t1	t2	t3	t4-tn	tn+1
IR_in	1				
ACC_in					
ALU[3:0]					
MDU[1:0]		00(有符号乘)		01(无符号乘)	10(有符号除)
MDU[1:0]					11(无符号除)
DM.L					
DM.W					
byte_out_c[2:0]					
byte_in_ena[3:0]					
HI.W			1		
LO.W			1		
M1					
M2					
M3					
M4[2:0]					
M5[2:0]	010 (PC+4)				
M6			1 (Res_HI)		
M7			1 (Res_LO)		
mfc0					
eret					
exception_cause[4:0]					
MDR_in					
Reg_in					
extc					
RsReg_in		1			
RtReg_in		1			
MDU_start			1	1	1
Latch_PC_in					
M4					

(3) 定义 54 条指令所有所需的状态，写出状态转移逻辑

状态定义：

localparam S_IF = 5'd0; // 00000: 取指
localparam S_ID = 5'd1; // 00001: 译码
localparam S_EX_ALU = 5'd2; // 00010: ALU 执行
localparam S_WB_ALU = 5'd3; // 00011: ALU 结果写回
localparam S_EX_ADDR = 5'd4; // 00100: 计算访存地址
localparam S_MEM_READ = 5'd5; // 00101: 读数据存储器
localparam S_MEM_WRITE = 5'd6; // 00110: 写数据存储器
localparam S_WB_LOAD = 5'd7; // 00111: 加载指令写回
localparam S_EX_BRANCH = 5'd8; // 01000: 执行分支
localparam S_EX_JUMP = 5'd9; // 01001: 执行跳转
localparam S_MDU_START = 5'd10; // 01010: 启动 MDU
localparam S_MDU_WAIT = 5'd11; // 01011: 等待 MDU
localparam S_MDU_DONE = 5'd12; // 01100: MDU 结果写入 HI/LO
localparam S_WB_FROM_HILO = 5'd13; // 01101: 从 HI/LO 写回 RegFile
localparam S_EX_TO_HILO = 5'd14; // 01110: 写入 HI/LO 寄存器
localparam S_EXCEPTION = 5'd15; // 01111: 异常处理

```
localparam S_EX_TO_CP0    = 5'd16; // 10000: 写入 CP0
localparam S_WB_FROM_CP0  = 5'd17; // 10001: 从 CP0 写回 RegFile
localparam S_EX_ERET      = 5'd18; // 10010: 执行 eret
```

状态转移机 FSM 逻辑:

```
// 状态机逻辑

always @(posedge clk or posedge rst) begin
    if (rst) begin
        state <= S_IF;
    end else begin
        state <= next_state;
    end
end

// 状态转移逻辑
always @(*) begin
    case (state)
        S_IF: next_state = S_ID;
        S_ID: begin
            case (op)
                6'b000000: begin // R 型指令
                    case (func)
                        6'b011000, 6'b011001, 6'b011010, 6'b011011: next_state = S_MDU_START;
                        6'b010000, 6'b010010: next_state = S_WB_FROM_HILO;
                        6'b010001, 6'b010011: next_state = S_EX_TO_HILO;
                        6'b001001, 6'b001000: next_state = S_EX_JUMP;
                        6'b001100, 6'b001101, 6'b110100: next_state = S_EXCEPTION;
                        default: next_state = S_EX_ALU;
                    endcase
                end
                6'b011100: next_state = (func == 6'b100000) ? S_EX_ALU : S_IF;
                6'b001000, 6'b001001, 6'b001100, 6'b001101, 6'b001010, 6'b001011, 6'b001111,
                6'b001110: next_state = S_EX_ALU;
                6'b100000, 6'b100001, 6'b100011, 6'b100100, 6'b100101, 6'b101000, 6'b101001,
                6'b101011: next_state = S_EX_ADDR;
                6'b000010, 6'b000011: next_state = S_EX_JUMP;
                6'b000100, 6'b000101, 6'b000001: next_state = S_EX_BRANCH;
                6'b010000: begin
                    case (rs)
                        5'b00000: next_state = S_WB_FROM_CP0;
                        5'b00100: next_state = S_EX_TO_CP0;
                        default: next_state = (func == 6'b011000) ? S_EX_ERET : S_IF;
                    endcase
                end
            endcase
        end
    end
```

```

        default: next_state = S_IF;
    endcase

end

S_EX_ALU: next_state = S_WB_ALU;
S_WB_ALU: next_state = S_IF;
S_EX_ADDR: next_state = (op[5:3] == 3'b100) ? S_MEM_READ : S_MEM_WRITE;
S_MEM_READ: next_state = S_WB_LOAD;
S_MEM_WRITE: next_state = S_IF;
S_WB_LOAD: next_state = S_IF;
S_EX_BRANCH: next_state = S_IF;
S_EX_JUMP: next_state = S_IF;
S_MDU_START: next_state = S_MDU_WAIT;
S_MDU_WAIT: next_state = MDU_busy ? S_MDU_WAIT : S_MDU_DONE;
S_MDU_DONE: next_state = S_IF;
S_WB_FROM_HILO: next_state = S_IF;
S_EX_TO_HILO: next_state = S_IF;
S_EXCEPTION: next_state = S_IF;
S_EX_TO_CP0: next_state = S_IF;
S_WB_FROM_CP0: next_state = S_IF;
S_EX_ERET: next_state = S_IF;
    default: next_state = S_IF;
endcase

end

```

(4) 根据指令操作时间表，写出每个控制信号的逻辑表达式。

PC_in = (state == S_WB_ALU) || (state == S_MEM_WRITE) || (state == S_WB_LOAD) || (state == S_EX_JUMP) || (state == S_EX_BRANCH) || (state == S_MDU_DONE) || (state == S_WB_FROM_HILO) || (state == S_EX_TO_HILO) || (state == S_WB_FROM_CP0) || (state == S_EX_TO_CP0) || (state == S_EX_ERET) || (state == S_EXCEPTION);

IR_in = (state == S_IF);

ACC_in = (state == S_EX_ALU) || (state == S_EX_ADDR);

RsReg_in = (state == S_ID);

RtReg_in = (state == S_ID);

DM_r = (state == S_MEM_READ);

DM_w = (state == S_MEM_WRITE);

MDR_in = (state == S_MEM_READ);

Reg_in = (state == S_WB_ALU) || (state == S_WB_LOAD) || (state == S_WB_FROM_HILO) || (state == S_WB_FROM_CP0) || (state == S_EX_JUMP && (op == 6'b000011 || (op == 6'b000000 && func == 6'b001001)));

HI_w = (state == S_MDU_DONE) || (state == S_EX_TO_HILO && op == 6'b000000 && func == 6'b010001);

LO_w = (state == S_MDU_DONE) || (state == S_EX_TO_HILO && op == 6'b000000 && func == 6'b010011);

MDU_start = (state == S_MDU_START);

mfc0 = (state == S_ID && op == 6'b010000 && rs == 5'b000000) || (state == S_WB_FROM_CP0);

mtc0 = (state == S_EX_TO_CP0);

eret = (state == S_EX_ERET);

exception = (state == S_EXCEPTION);

Latch_PC4_in_M4 = (state == S_ID && (op == 6'b000011 || (op == 6'b000000 && func == 6'b001001)));

M1 = (state == S_EX_ALU && op == 6'b000000 && (func == 6'b000000 || func == 6'b000010 || func == 6'b000011));

M2 = (state == S_WB_ALU && op != 6'b000000 && op != 6'b011100) || (state == S_WB_LOAD) || (state == S_WB_FROM_CP0);

M3 = (state == S_EX_ALU && op != 6'b000000 && op != 6'b011100) || (state == S_EX_ADDR);

EXT_c = (state == S_EX_ALU && (op == 6'b001000 || op == 6'b001001 || op == 6'b001010 || op == 6'b001011)) || (state == S_EX_ADDR);

M6 = (state == S_MDU_DONE) || (state == S_EX_TO_HILO && op == 6'b000000 && func == 6'b010001);

M7 = (state == S_MDU_DONE) || (state == S_EX_TO_HILO && op == 6'b000000 && func == 6'b010011);

M4 = (state == S_WB_FROM_CP0) ? 3'b000 : // CP0 数据到寄存器
(state == S_WB_FROM_HILO && op == 6'b000000 && func ==

```

6'b010000) ? 3'b001 : // HI 到寄存器
    (state == S_WB_FROM_HILO && op == 6'b000000 && func ==
6'b010010) ? 3'b010 : // LO 到寄存器
    (state == S_EX_JUMP && (op == 6'b000011 || (op == 6'b000000 &&
func == 6'b001001))) ? 3'b011 : // PC+4 到寄存器
    (state == S_WB_LOAD) ? 3'b100 : // MDR 到寄存器
    (state == S_WB_ALU) ? 3'b101 : // ACC 到寄存器
    3'b010; // 默认值

```

```

M5 = (state == S_EX_JUMP && op == 6'b000000 && (func == 6'b001000 || func ==
6'b001001)) ? 3'b000 : // Rs 寄存器值
    (state == S_EX_BRANCH && ((op == 6'b000100 && ZF) || (op ==
6'b000101 && !ZF) ||
    (op == 6'b000001 && rt == 5'b00001 && !SF))) ? 3'b001 : // 分支
跳转
    (state == S_EX_ERET || state == S_EXCEPTION) ? 3'b011 : // 异常
地址
    (state == S_EX_JUMP && (op == 6'b000010 || op == 6'b000011)) ?
3'b100 : // 跳转地址
    3'b010; // PC+4

```

```

ALUc = (state == S_EX_ALU && op == 6'b000000 && func == 6'b100000) ?
4'b0010 : // add
    (state == S_EX_ALU && op == 6'b000000 && func == 6'b100001) ?
4'b0000 : // addu
    (state == S_EX_ALU && op == 6'b000000 && func == 6'b100010) ?
4'b0011 : // sub
    (state == S_EX_ALU && op == 6'b000000 && func == 6'b100011) ?
4'b0001 : // subu
    (state == S_EX_ALU && op == 6'b000000 && func == 6'b100100) ?
4'b0100 : // and
    (state == S_EX_ALU && op == 6'b000000 && func == 6'b100101) ?
4'b0101 : // or
    (state == S_EX_ALU && op == 6'b000000 && func == 6'b100110) ?
4'b0110 : // xor
    (state == S_EX_ALU && op == 6'b000000 && func == 6'b100111) ?
4'b0111 : // nor
    (state == S_EX_ALU && op == 6'b000000 && func == 6'b101010) ?
4'b1011 : // slt
    (state == S_EX_ALU && op == 6'b000000 && func == 6'b101011) ?
4'b1010 : // sltu
    (state == S_EX_ALU && op == 6'b000000 && func == 6'b000000) ?
4'b1110 : // sll
    (state == S_EX_ALU && op == 6'b000000 && func == 6'b000010) ?

```

```

4'b1101 : // srl
                (state == S_EX_ALU && op == 6'b000000 && func == 6'b000011) ?
4'b1100 : // sra
                (state == S_EX_ALU && op == 6'b000000 && func == 6'b000100) ?
4'b1110 : // sllv
                (state == S_EX_ALU && op == 6'b000000 && func == 6'b000110) ?
4'b1101 : // srlv
                (state == S_EX_ALU && op == 6'b000000 && func == 6'b000111) ?
4'b1100 : // srav
                (state == S_EX_ALU && op == 6'b001000) ? 4'b0010 : // addi
                (state == S_EX_ALU && op == 6'b001001) ? 4'b0000 : // addiu
                (state == S_EX_ALU && op == 6'b001100) ? 4'b0100 : // andi
                (state == S_EX_ALU && op == 6'b001101) ? 4'b0101 : // ori
                (state == S_EX_ALU && op == 6'b001110) ? 4'b0110 : // xori
                (state == S_EX_ALU && op == 6'b001010) ? 4'b1011 : // slti
                (state == S_EX_ALU && op == 6'b001011) ? 4'b1010 : // sltiu
                (state == S_EX_ALU && op == 6'b001111) ? 4'b1000 : // lui
                (state == S_EX_ALU && op == 6'b011100 && func == 6'b100000) ?
4'b1111 : // clz
                (state == S_EX_ADDR) ? 4'b0000 : // addu for address calculation
                (state == S_EX_BRANCH && (op == 6'b000100 || op ==
6'b000101)) ? 4'b0011 : // sub for branch
                (state == S_EX_BRANCH && op == 6'b000001) ? 4'b1001 : //
bgez
                4'b0000; // 默认值

```

```

MDUc = (state == S_MDU_START && op == 6'b000000 && func == 6'b011000) ?
2'b00 : // mult
                (state == S_MDU_START && op == 6'b000000 && func ==
6'b011001) ? 2'b01 : // multu
                (state == S_MDU_START && op == 6'b000000 && func ==
6'b011010) ? 2'b10 : // div
                (state == S_MDU_START && op == 6'b000000 && func ==
6'b011011) ? 2'b11 : // divu
                (state == S_MDU_WAIT && op == 6'b000000 && func ==
6'b011000) ? 2'b00 : // mult
                (state == S_MDU_WAIT && op == 6'b000000 && func ==
6'b011001) ? 2'b01 : // multu
                (state == S_MDU_WAIT && op == 6'b000000 && func ==
6'b011010) ? 2'b10 : // div
                (state == S_MDU_WAIT && op == 6'b000000 && func ==
6'b011011) ? 2'b11 : // divu
                2'b00; // 默认值

```

```

byte_out_c = (state == S_MEM_READ && op == 6'b100000) ? 3'b001 : // lb
              (state == S_MEM_READ && op == 6'b100100) ? 3'b010 : //
lb
              (state == S_MEM_READ && op == 6'b100001) ? 3'b011 : //
lh
              (state == S_MEM_READ && op == 6'b100101) ? 3'b100 : //
lhu
              (state == S_MEM_READ && op == 6'b100011) ? 3'b000 : //
lw
              3'b000; // 默认值

byte_in_ena = (state == S_MEM_WRITE && op == 6'b101011) ? 4'b1111 : // sw
              (state == S_MEM_WRITE && op == 6'b101001) ?
4'b0011 : // sh
              (state == S_MEM_WRITE && op == 6'b101000) ?
4'b0001 : // sb
              4'b0000; // 默认值

cause = (state == S_EXCEPTION && op == 6'b000000 && func == 6'b001100) ?
5'b01000 : // syscall
        (state == S_EXCEPTION && op == 6'b000000 && func ==
6'b001101) ? 5'b01001 : // break
        (state == S_EXCEPTION && op == 6'b000000 && func ==
6'b110100) ? 5'b01101 : // teq
        5'b00000; // 默认值

```

(5) 根据状态转移逻辑和控制信号的逻辑表达式，完成控制部件 CU 的设计。

```

module CU(
    input clk,
    input rst,

    input [31:0] Instr, // 输入指令
    input [31:0] ACC, // 输入 ACC 寄存器值
    input ZF, // 零标志
    input SF, // 符号标志
    input [31:0] CP0_status, // CP0 状态寄存器值
    input MDU_busy, // MDU 忙信号
    // 控制信号输出
    output PC_in, // PC 写使能
    output IR_in, // 指令寄存器写使能
    output ACC_in, // ACC 寄存器写使能
    output [3:0] ALUc, // ALU 控制信号
    output [1:0] MDUc, // MDU 控制信号
    output DM_r, // 数据存储器读使能

```

```

output DM_w,          // 数据存储器写使能
output [2:0] byte_out_c, // 字节输出控制信号
output [3:0] byte_in_ena, // 字节输入使能信号
output HI_w,          // HI 寄存器写使能
output LO_w,          // LO 寄存器写使能
output M1,            // 多路选择器 M1 控制信号
output M2,            // 多路选择器 M2 控制信号
output M3,            // 多路选择器 M3 控制信号
output [2:0] M4,       // 多路选择器 M4 控制信号
output [2:0] M5,       // 多路选择器 M5 控制信号
output M6,            // 多路选择器 M6 控制信号
output M7,            // 多路选择器 M7 控制信号
output mfc0,          // CP0 读控制信号
output mtc0,          // CP0 写控制信号
output eret,          // 异常返回信号
output exception,     // 异常信号
output [4:0] cause,    // 异常原因码
output MDR_in,        // MDR 寄存器写使能
output EXT_c,         // 扩展控制信号
output Reg_in,        // 寄存器堆写使能
output RsReg_in,      // Rs 寄存器写使能
output RtReg_in,      // Rt 寄存器写使能
output MDU_start,     // MDU 启动信号
output Latch_PC4_in_M4 // PC4 寄存器写使能(MUX4)
);

// 指令字段解析
wire [5:0] op = Instr[31:26]; // 操作码
wire [4:0] rs = Instr[25:21]; // 源寄存器 1
wire [4:0] rt = Instr[20:16]; // 源寄存器 2
wire [4:0] rd = Instr[15:11]; // 目标寄存器
wire [4:0] shamt = Instr[10:6]; // 移位量
wire [5:0] func = Instr[5:0]; // 功能码
wire [15:0] imm = Instr[15:0]; // 立即数
wire [25:0] addr = Instr[25:0]; // 跳转地址

// 状态定义
localparam S_IF      = 5'd0; // 00000: 取指
localparam S_ID      = 5'd1; // 00001: 译码
localparam S_EX_ALU   = 5'd2; // 00010: ALU 执行
localparam S_WB_ALU   = 5'd3; // 00011: ALU 结果写回
localparam S_EX_ADDR  = 5'd4; // 00100: 计算访存地址
localparam S_MEM_READ = 5'd5; // 00101: 读数据存储器
localparam S_MEM_WRITE = 5'd6; // 00110: 写数据存储器
localparam S_WB_LOAD  = 5'd7; // 00111: 加载指令写回

```

```

localparam S_EX_BRANCH    = 5'd8; // 01000: 执行分支
localparam S_EX_JUMP      = 5'd9; // 01001: 执行跳转
localparam S_MDU_START    = 5'd10; // 01010: 启动 MDU
localparam S_MDU_WAIT     = 5'd11; // 01011: 等待 MDU
localparam S_MDU_DONE     = 5'd12; // 01100: MDU 结果写入 HI/LO
localparam S_WB_FROM_HILO = 5'd13; // 01101: 从 HI/LO 写回 RegFile
localparam S_EX_TO_HILO   = 5'd14; // 01110: 写入 HI/LO 寄存器
localparam S_EXCEPTION    = 5'd15; // 01111: 异常处理
localparam S_EX_TO_CP0    = 5'd16; // 10000: 写入 CP0
localparam S_WB_FROM_CP0  = 5'd17; // 10001: 从 CP0 写回 RegFile
localparam S_EX_ERET      = 5'd18; // 10010: 执行 eret

// 状态寄存器
reg [4:0] state;
reg [4:0] next_state;

// 状态机逻辑
always @(posedge clk or posedge rst) begin
    if (rst) begin
        state <= S_IF;
    end else begin
        state <= next_state;
    end
end

// 状态转移逻辑
always @(*) begin
    case (state)
        S_IF: next_state = S_ID;
        S_ID: begin
            case (op)
                6'b000000: begin // R 型指令
                    case (func)
                        6'b011000, 6'b011001, 6'b011010, 6'b011011: next_state = S_MDU_START;
                        6'b010000, 6'b010010: next_state = S_WB_FROM_HILO;
                        6'b010001, 6'b010011: next_state = S_EX_TO_HILO;
                        6'b001001, 6'b001000: next_state = S_EX_JUMP;
                        6'b001100, 6'b001101, 6'b110100: next_state = S_EXCEPTION;
                        default: next_state = S_EX_ALU;
                    endcase
                end
                6'b011100: next_state = (func == 6'b100000) ? S_EX_ALU : S_IF;
                6'b001000, 6'b001001, 6'b001100, 6'b001101, 6'b001010, 6'b001011, 6'b001111,
                6'b001110: next_state = S_EX_ALU;
                6'b100000, 6'b100001, 6'b100011, 6'b100100, 6'b100101, 6'b101000, 6'b101001,
                6'b101011: next_state = S_EX_ADDR;
            endcase
        end
    endcase
end

```

```

        6'b000010, 6'b000011: next_state = S_EX_JUMP;
        6'b000100, 6'b000101, 6'b000001: next_state = S_EX_BRANCH;
        6'b010000: begin
            case (rs)
                5'b00000: next_state = S_WB_FROM_CP0;
                5'b00100: next_state = S_EX_TO_CP0;
                default: next_state = (func == 6'b011000) ? S_EX_ERET : S_IF;
            endcase
        end
        default: next_state = S_IF;
    endcase
end

S_EX_ALU: next_state = S_WB_ALU;
S_WB_ALU: next_state = S_IF;
S_EX_ADDR: next_state = (op[5:3] == 3'b100) ? S_MEM_READ : S_MEM_WRITE;
S_MEM_READ: next_state = S_WB_LOAD;
S_MEM_WRITE: next_state = S_IF;
S_WB_LOAD: next_state = S_IF;
S_EX_BRANCH: next_state = S_IF;
S_EX_JUMP: next_state = S_IF;
S_MDU_START: next_state = S_MDU_WAIT;
S_MDU_WAIT: next_state = MDU_busy ? S_MDU_WAIT : S_MDU_DONE;
S_MDU_DONE: next_state = S_IF;
S_WB_FROM_HILO: next_state = S_IF;
S_EX_TO_HILO: next_state = S_IF;
S_EXCEPTION: next_state = S_IF;
S_EX_TO_CP0: next_state = S_IF;
S_WB_FROM_CP0: next_state = S_IF;
S_EX_ERET: next_state = S_IF;
default: next_state = S_IF;
endcase
end

// 组合逻辑控制信号生成
// PC_in: PC 写使能
assign PC_in = (state == S_WB_ALU) || (state == S_MEM_WRITE) || (state == S_WB_LOAD) ||
               (state == S_EX_JUMP) || (state == S_EX_BRANCH) || (state == S_MDU_DONE) ||
               (state == S_WB_FROM_HILO) || (state == S_EX_TO_HILO) || (state == S_WB_FROM_CP0) ||
               (state == S_EX_TO_CP0) || (state == S_EX_ERET) || (state == S_EXCEPTION);

// IR_in: 指令寄存器写使能
assign IR_in = (state == S_IF);

// ACC_in: ACC 寄存器写使能
assign ACC_in = (state == S_EX_ALU) || (state == S_EX_ADDR);

// RsReg_in: Rs 寄存器写使能
assign RsReg_in = (state == S_ID);

```

```

// RtReg_in: Rt 寄存器写使能
assign RtReg_in = (state == S_ID);

// DM_r: 数据存储器读使能
assign DM_r = (state == S_MEM_READ);

// DM_w: 数据存储器写使能
assign DM_w = (state == S_MEM_WRITE);

// MDR_in: MDR 寄存器写使能
assign MDR_in = (state == S_MEM_READ);

// Reg_in: 寄存器堆写使能
assign Reg_in = (state == S_WB_ALU) || (state == S_WB_LOAD) || (state == S_WB_FROM_HILO) ||
                (state == S_WB_FROM_CP0) || (state == S_EX_JUMP && (op == 6'b000011 ||
                (op == 6'b000000 && func == 6'b001001)));

// HI_w: HI 寄存器写使能
assign HI_w = (state == S_MDU_DONE) || (state == S_EX_TO_HILO && op == 6'b000000 && func == 6'b010001);

// LO_w: LO 寄存器写使能
assign LO_w = (state == S_MDU_DONE) || (state == S_EX_TO_HILO && op == 6'b000000 && func == 6'b010011);

// MDU_start: MDU 启动信号
assign MDU_start = (state == S_MDU_START);

// mfc0: CP0 读控制信号
assign mfc0 = (state == S_ID && op == 6'b010000 && rs == 5'b00000) ||
                (state == S_WB_FROM_CP0);

// mtc0: CP0 写控制信号
assign mtc0 = (state == S_EX_TO_CP0);

// eret: 异常返回信号
assign eret = (state == S_EX_ERET);

// exception: 异常信号
assign exception = (state == S_EXCEPTION);

// Latch_PC4_in_M4: PC4 寄存器写使能
assign Latch_PC4_in_M4 = (state == S_ID && (op == 6'b000011 ||
                (op == 6'b000000 && func == 6'b001001)));

// M1: 多路选择器 M1 控制信号
assign M1 = (state == S_EX_ALU && op == 6'b000000 &&
                (func == 6'b000000 || func == 6'b000010 || func == 6'b000011));

// M2: 多路选择器 M2 控制信号
assign M2 = (state == S_WB_ALU && op != 6'b000000 && op != 6'b011100) ||
                (state == S_WB_LOAD) || (state == S_WB_FROM_CP0);

// M3: 多路选择器 M3 控制信号
assign M3 = (state == S_EX_ALU && op != 6'b000000 && op != 6'b011100) ||
                (state == S_EX_ADDR);

// EXT_c: 扩展控制信号
assign EXT_c = (state == S_EX_ALU && (op == 6'b001000 || op == 6'b001001 ||
                op == 6'b001010 || op == 6'b001011)) || (state == S_EX_ADDR);

// M6: 多路选择器 M6 控制信号
assign M6 = (state == S_MDU_DONE) || (state == S_EX_TO_HILO && op == 6'b000000 && func == 6'b010001);

```

```

// M7: 多路选择器 M7 控制信号
assign M7 = (state == S_MDU_DONE) || (state == S_EX_TO_HILO && op == 6'b000000 && func == 6'b010011);

// M4: 多路选择器 M4 控制信号 (3 位)
assign M4 = (state == S_WB_FROM_CP0) ? 3'b000 : // CP0 数据到寄存器

        (state == S_WB_FROM_HILO && op == 6'b000000 && func == 6'b010000) ? 3'b001 : // HI 到寄存器

        (state == S_WB_FROM_HILO && op == 6'b000000 && func == 6'b010010) ? 3'b010 : // LO 到寄存器

        (state == S_EX_JUMP && (op == 6'b000011 || (op == 6'b000000 && func == 6'b001001))) ?
3'b011 : // PC+4 到寄存器

        (state == S_WB_LOAD) ? 3'b100 : // MDR 到寄存器

        (state == S_WB_ALU) ? 3'b101 : // ACC 到寄存器

        3'b010; // 默认值

// M5: 多路选择器 M5 控制信号 (3 位)
assign M5 = (state == S_EX_JUMP && op == 6'b000000 && (func == 6'b001000 || func == 6'b001001)) ?
3'b000 : // Rs 寄存器值

        (state == S_EX_BRANCH && ((op == 6'b000100 && ZF) || (op == 6'b000101 && !ZF) ||
        (op == 6'b000001 && rt == 5'b00001 && !SF))) ? 3'b001 : // 分支跳转

        (state == S_EX_ERET || state == S_EXCEPTION) ? 3'b011 : // 异常地址

        (state == S_EX_JUMP && (op == 6'b000010 || op == 6'b000011)) ? 3'b100 : // 跳转地址

        3'b010; // PC+4

// ALUc: ALU 控制信号 (4 位)
assign ALUc = (state == S_EX_ALU && op == 6'b000000 && func == 6'b100000) ? 4'b0010 : // add
        (state == S_EX_ALU && op == 6'b000000 && func == 6'b100001) ? 4'b0000 : // addu
        (state == S_EX_ALU && op == 6'b000000 && func == 6'b100010) ? 4'b0011 : // sub
        (state == S_EX_ALU && op == 6'b000000 && func == 6'b100011) ? 4'b0001 : // subu
        (state == S_EX_ALU && op == 6'b000000 && func == 6'b100100) ? 4'b0100 : // and
        (state == S_EX_ALU && op == 6'b000000 && func == 6'b100101) ? 4'b0101 : // or
        (state == S_EX_ALU && op == 6'b000000 && func == 6'b100110) ? 4'b0110 : // xor
        (state == S_EX_ALU && op == 6'b000000 && func == 6'b100111) ? 4'b0111 : // nor
        (state == S_EX_ALU && op == 6'b000000 && func == 6'b101010) ? 4'b1011 : // slt
        (state == S_EX_ALU && op == 6'b000000 && func == 6'b101011) ? 4'b1010 : // sltu
        (state == S_EX_ALU && op == 6'b000000 && func == 6'b000000) ? 4'b1110 : // sll
        (state == S_EX_ALU && op == 6'b000000 && func == 6'b000010) ? 4'b1101 : // srl
        (state == S_EX_ALU && op == 6'b000000 && func == 6'b000011) ? 4'b1100 : // sra
        (state == S_EX_ALU && op == 6'b000000 && func == 6'b000100) ? 4'b1110 : // sllv
        (state == S_EX_ALU && op == 6'b000000 && func == 6'b000110) ? 4'b1101 : // srlv
        (state == S_EX_ALU && op == 6'b000000 && func == 6'b000111) ? 4'b1100 : // srav
        (state == S_EX_ALU && op == 6'b001000) ? 4'b0010 : // addi
        (state == S_EX_ALU && op == 6'b001001) ? 4'b0000 : // addiu
        (state == S_EX_ALU && op == 6'b001100) ? 4'b0100 : // andi
        (state == S_EX_ALU && op == 6'b001101) ? 4'b0101 : // ori
        (state == S_EX_ALU && op == 6'b001110) ? 4'b0110 : // xori
        (state == S_EX_ALU && op == 6'b001010) ? 4'b1011 : // slti

```

```

        (state == S_EX_ALU && op == 6'b001011) ? 4'b1010 : // sltiu
        (state == S_EX_ALU && op == 6'b001111) ? 4'b1000 : // lui
        (state == S_EX_ALU && op == 6'b011100 && func == 6'b100000) ? 4'b1111 : // clz
        (state == S_EX_ADDR) ? 4'b0000 : // addu for address calculation
        (state == S_EX_BRANCH && (op == 6'b000100 || op == 6'b000101)) ? 4'b0011 : // sub for
branch

        (state == S_EX_BRANCH && op == 6'b000001) ? 4'b1001 : // bgez
        4'b0000; // 默认值

// MDUc: MDU 控制信号 (2 位)
assign MDUc = (state == S_MDU_START && op == 6'b000000 && func == 6'b011000) ? 2'b00 : // mult
        (state == S_MDU_START && op == 6'b000000 && func == 6'b011001) ? 2'b01 : // multu
        (state == S_MDU_START && op == 6'b000000 && func == 6'b011010) ? 2'b10 : // div
        (state == S_MDU_START && op == 6'b000000 && func == 6'b011011) ? 2'b11 : // divu
        (state == S_MDU_WAIT && op == 6'b000000 && func == 6'b011000) ? 2'b00 : // mult
        (state == S_MDU_WAIT && op == 6'b000000 && func == 6'b011001) ? 2'b01 : // multu
        (state == S_MDU_WAIT && op == 6'b000000 && func == 6'b011010) ? 2'b10 : // div
        (state == S_MDU_WAIT && op == 6'b000000 && func == 6'b011011) ? 2'b11 : // divu
        2'b00; // 默认值

// byte_out_c: 字节输出控制信号 (3 位)
assign byte_out_c = (state == S_MEM_READ && op == 6'b100000) ? 3'b001 : // lb
        (state == S_MEM_READ && op == 6'b100100) ? 3'b010 : // lbu
        (state == S_MEM_READ && op == 6'b100001) ? 3'b011 : // lh
        (state == S_MEM_READ && op == 6'b100101) ? 3'b100 : // lhu
        (state == S_MEM_READ && op == 6'b100011) ? 3'b000 : // lw
        3'b000; // 默认值

// byte_in_ena: 字节输入使能信号 (4 位)
assign byte_in_ena = (state == S_MEM_WRITE && op == 6'b101011) ? 4'b1111 : // sw
        (state == S_MEM_WRITE && op == 6'b101001) ? 4'b0011 : // sh
        (state == S_MEM_WRITE && op == 6'b101000) ? 4'b0001 : // sb
        4'b0000; // 默认值

// cause: 异常原因码 (5 位)
assign cause = (state == S_EXCEPTION && op == 6'b000000 && func == 6'b001100) ? 5'b01000 : // syscall
        (state == S_EXCEPTION && op == 6'b000000 && func == 6'b001101) ? 5'b01001 : // break
        (state == S_EXCEPTION && op == 6'b000000 && func == 6'b110100) ? 5'b01101 : // teq
        5'b00000; // 默认值

endmodule

```

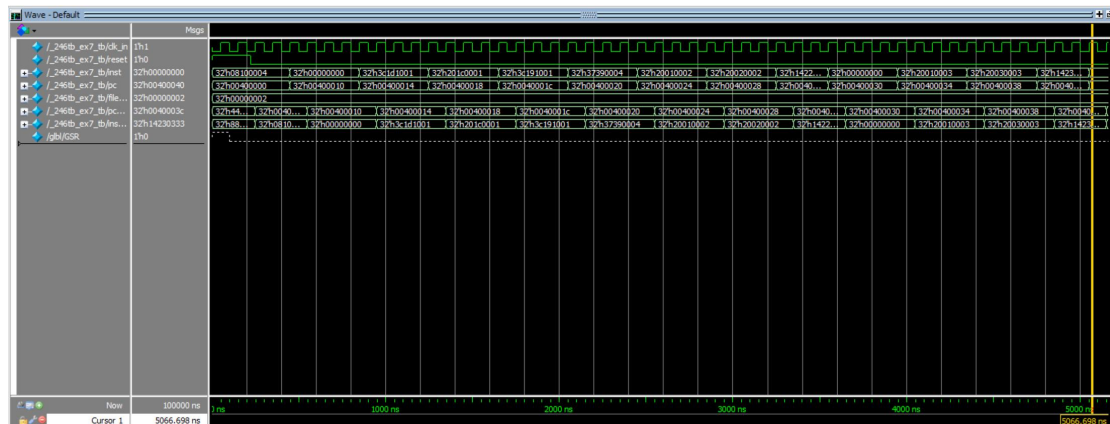
四、CPU 前仿真测试结果

(1) 对 CPU 前仿真的实验内容和结果进行描述，并给出 modelsim 前仿真波形图的贴图。

前仿真也称为功能仿真，主旨在于验证电路的功能是否符合设计要求，其特点是不考虑电路门延迟与线延迟，主要是验证电路与理想情况是否一致。可综合

FPGA 代码是用 RTL 级代码语言描述的，其输入为 RTL 级代码与 Testbench.

前仿真可以使用 iram，也可以直接用 coe 文件配置 IP 核，我采用的是后者。在 MIPS 网站上交时，我将下发的 coe 文件配置到我的 IP 核，也是成功 AC 了，可以证明 CPU 的设计是没问题的。



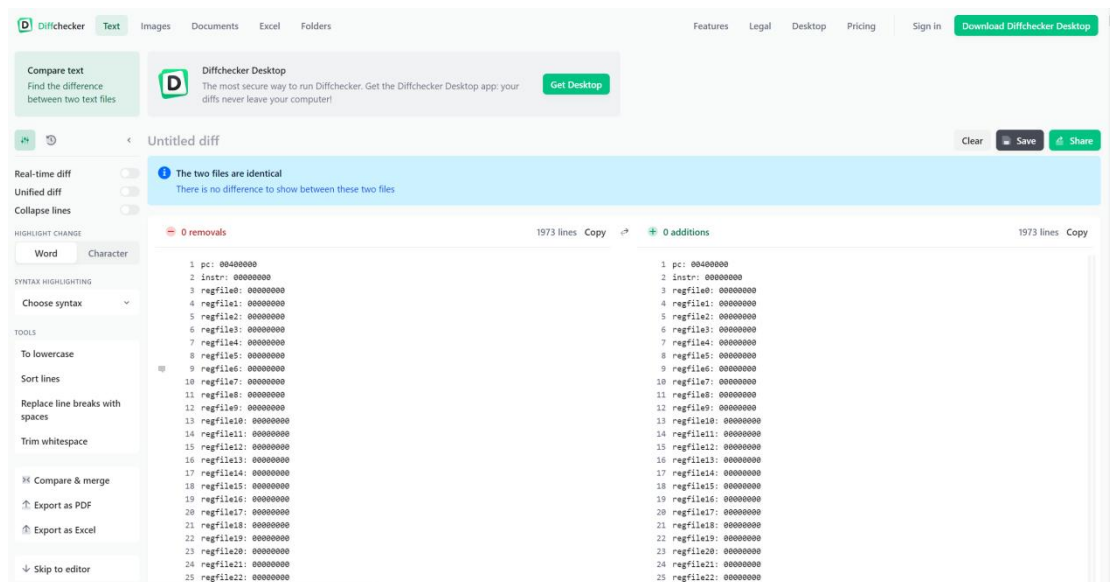
(2) 对单条指令、网站前仿真所用测试代码 coe 文件等测试的结果进行比对。

用 Mars 运行 MIPS 汇编程序后会在 Mars 目录下生成一个 result.txt 文件

用自己的 CPU 运行导出的 MIPS 汇编程序，将 pc、instr、regfile 结果输出到文件，在 CPU 工程目录下生成另一个 result.txt 文件。

将两个文件进行比对。下面放了一些指令的对比结果，不论是寄存器堆中的值，还是数据存储器的值，都和 Mars 运行的结果相同，可以验证设计的 CPU 是没问题的。

addi



addiu

Clear Save Share

0 removals	1973 lines	Copy	↔	0 additions	1973 lines	Copy
1	pc: 00400000			1	pc: 00400000	
2	instr: 00000000			2	instr: 00000000	
3	regfile0: 00000000			3	regfile0: 00000000	
4	regfile1: 00000000			4	regfile1: 00000000	
5	regfile2: 00000000			5	regfile2: 00000000	
6	regfile3: 00000000			6	regfile3: 00000000	
7	regfile4: 00000000			7	regfile4: 00000000	
8	regfile5: 00000000			8	regfile5: 00000000	
9	regfile6: 00000000			9	regfile6: 00000000	
10	regfile7: 00000000			10	regfile7: 00000000	
11	regfile8: 00000000			11	regfile8: 00000000	
12	regfile9: 00000000			12	regfile9: 00000000	
13	regfile10: 00000000			13	regfile10: 00000000	
14	regfile11: 00000000			14	regfile11: 00000000	
15	regfile12: 00000000			15	regfile12: 00000000	
16	regfile13: 00000000			16	regfile13: 00000000	
17	regfile14: 00000000			17	regfile14: 00000000	
18	regfile15: 00000000			18	regfile15: 00000000	
19	regfile16: 00000000			19	regfile16: 00000000	
20	regfile17: 00000000			20	regfile17: 00000000	
21	regfile18: 00000000			21	regfile18: 00000000	
22	regfile19: 00000000			22	regfile19: 00000000	
23	regfile20: 00000000			23	regfile20: 00000000	
24	regfile21: 00000000			24	regfile21: 00000000	
25	regfile22: 00000000			25	regfile22: 00000000	
26	regfile23: 00000000			26	regfile23: 00000000	

Untitled diff

Clear Save Share

0 removals
1123 lines
Copy
+0 additions

```

1 pc: 00400000
2 instr: 00000000
3 regfile0: 00000000
4 regfile1: 00000000
5 regfile2: 00000000
6 regfile3: 00000000
7 regfile4: 00000000
8 regfile5: 00000000
9 regfile6: 00000000
10 regfile7: 00000000
11 regfile8: 00000000
12 regfile9: 00000000
13 regfile10: 00000000
14 regfile11: 00000000
15 regfile12: 00000000
16 regfile13: 00000000
17 regfile14: 00000000
18 regfile15: 00000000
19 regfile16: 00000000
20 regfile17: 00000000
21 regfile18: 00000000
22 regfile19: 00000000
23 regfile20: 00000000
24 regfile21: 00000000
25 regfile22: 00000000
26 regfile23: 00000000

```

Untitled diff

Clear Save Share Explain

3468 removals	6801 lines Copy	+ 0 additions	3333 lines Copy
<pre> 1 pc: 00400000 2 instr: 00000000 3 regfile0: 00000000 4 regfile1: 00000000 5 regfile2: 00000000 6 regfile3: 00000000 7 regfile4: 00000000 8 regfile5: 00000000 9 regfile6: 00000000 10 regfile7: 00000000 11 regfile8: 00000000 12 regfile9: 00000000 13 regfile10: 00000000 14 regfile11: 00000000 15 regfile12: 00000000 16 regfile13: 00000000 17 regfile14: 00000000 18 regfile15: 00000000 19 regfile16: 00000000 20 regfile17: 00000000 21 regfile18: 00000000 22 regfile19: 00000000 23 regfile20: 00000000 24 regfile21: 00000000 25 regfile22: 00000000 26 regfile23: 00000000 27 regfile24: 00000000 28 regfile25: 00000000 29 regfile26: 00000000 30 regfile27: 00000000 </pre>			<pre> 1 pc: 00400000 2 instr: 00000000 3 regfile0: 00000000 4 regfile1: 00000000 5 regfile2: 00000000 6 regfile3: 00000000 7 regfile4: 00000000 8 regfile5: 00000000 9 regfile6: 00000000 10 regfile7: 00000000 11 regfile8: 00000000 12 regfile9: 00000000 13 regfile10: 00000000 14 regfile11: 00000000 15 regfile12: 00000000 16 regfile13: 00000000 17 regfile14: 00000000 18 regfile15: 00000000 19 regfile16: 00000000 20 regfile17: 00000000 21 regfile18: 00000000 22 regfile19: 00000000 23 regfile20: 00000000 24 regfile21: 00000000 25 regfile22: 00000000 26 regfile23: 00000000 27 regfile24: 00000000 28 regfile25: 00000000 29 regfile26: 00000000 30 regfile27: 00000000 </pre>

Data Segment								
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x00000000	0xffffffff	0x0000000f	0x000000ff	0x00000fff	0xffffffff	0x00005555	0xffffffff
0x10010020	0xffffffff	0xffffffff	0xffffffff	0xffffffff	0xffffffff	0xffffffff	0x0000f000	0x00ff0000
0x10010040	0x00ff0000	0xffffffff	0x55550000	0xaaaa0000	0xbbbb0000	0xcccc0000	0xdddd0000	0x00000000
0x10010060	0xffff0000	0xffff0000	0x000f0000	0x00ff0000	0x0fff0000	0xffff0000	0x55550000	0x000000ff
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010140	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010160	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010180	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

mem [31:0]	3	0000000f...	Array
[31]	[31:0]	0000000f	Array
[30]	[31:0]	55550000	Array
[29]	[31:0]	ffff0000	Array
[28]	[31:0]	0fff0000	Array
[27]	[31:0]	00ff0000	Array
[26]	[31:0]	000f0000	Array
[25]	[31:0]	ffff0000	Array
[24]	[31:0]	ffff0000	Array
[23]	[31:0]	eeee0000	Array
[22]	[31:0]	ddd00000	Array
[21]	[31:0]	ccc00000	Array
[20]	[31:0]	bbb00000	Array
[19]	[31:0]	aaa00000	Array
[18]	[31:0]	55550000	Array
[17]	[31:0]	ffff0000	Array
[16]	[31:0]	0fff0000	Array
[15]	[31:0]	00ff0000	Array
[14]	[31:0]	000f0000	Array
[13]	[31:0]	ffff0000	Array
[12]	[31:0]	fffffff	Array
[11]	[31:0]	ffffeee	Array
[10]	[31:0]	ffffddd	Array
[9]	[31:0]	ffffccc	Array
[8]	[31:0]	ffffbbb	Array
[7]	[31:0]	ffffaaa	Array
[6]	[31:0]	00005555	Array
[5]	[31:0]	fffffff	Array
[4]	[31:0]	0000fff	Array
[3]	[31:0]	00000ff	Array
[2]	[31:0]	000000f	Array
[1]	[31:0]	fffffff	Array
[0]	[31:0]	0000000	Array

lws2

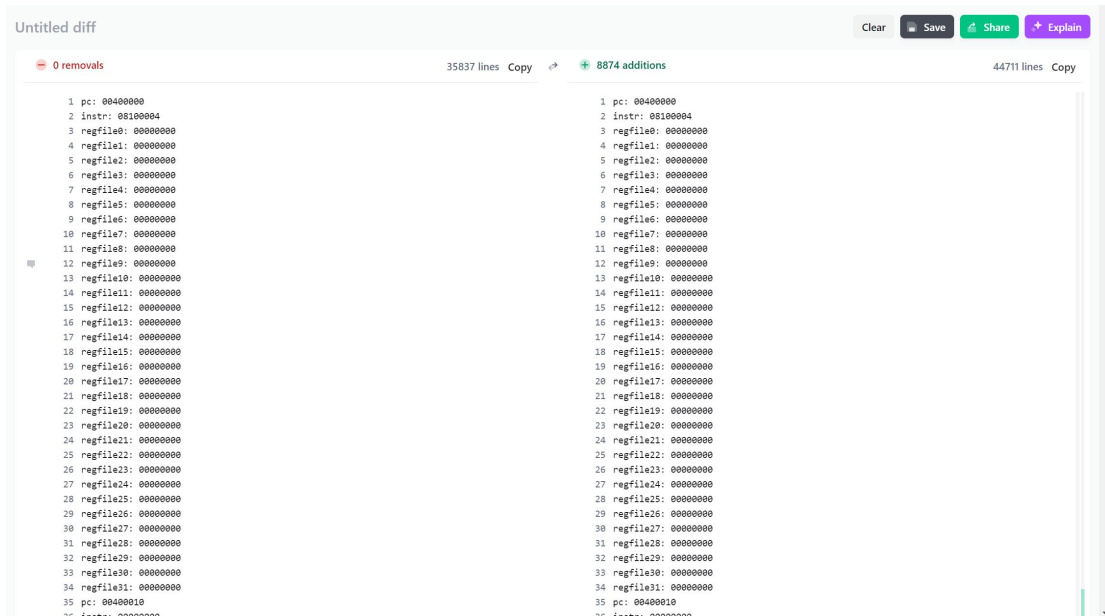
Untitled diff		Clear	Save	Share	Explain
3468 removals	6801 lines	Copy	0 additions	3333 lines	Copy
1 pc: 00400000	2 instr: 00000000	3 regfile0: 00000000	4 regfile1: 00000000	5 regfile2: 00000000	6 regfile3: 00000000
7 regfile4: 00000000	8 regfile5: 00000000	9 regfile6: 00000000	10 regfile7: 00000000	11 regfile8: 00000000	12 regfile9: 00000000
13 regfile10: 00000000	14 regfile11: 00000000	15 regfile12: 00000000	16 regfile13: 00000000	17 regfile14: 00000000	18 regfile15: 00000000
19 regfile16: 00000000	20 regfile17: 00000000	21 regfile18: 00000000	22 regfile19: 00000000	23 regfile20: 00000000	24 regfile21: 00000000
25 regfile22: 00000000	26 regfile23: 00000000	27 regfile24: 00000000	28 regfile25: 00000000	29 regfile26: 00000000	30 regfile27: 00000000

Data Segment								
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x00000000	0x00000001	0x00000002	0x00000003	0x00000004	0x00000005	0x00000006	0x00000007
0x10010020	0x00000008	0x00000009	0x0000000a	0x0000000b	0x0000000c	0x0000000d	0x0000000e	0x0000000f
0x10010040	0x00000010	0x00000011	0x00000012	0x00000013	0x00000014	0x00000015	0x00000016	0x00000017
0x10010060	0x00000018	0x00000019	0x0000001a	0x0000001b	0x0000001c	0x0000001d	0x0000001e	0x0000001f
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010140	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010160	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010180	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100101a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

mem[31:0][31:0]				Array
[31][31:0]	0000001f			Array
[30][31:0]	0000001e			Array
[29][31:0]	0000001d			Array
[28][31:0]	0000001c			Array
[27][31:0]	0000001b			Array
[26][31:0]	0000001a			Array
[25][31:0]	00000019			Array
[24][31:0]	00000018			Array
[23][31:0]	00000017			Array
[22][31:0]	00000016			Array
[21][31:0]	00000015			Array
[20][31:0]	00000014			Array
[19][31:0]	00000013			Array
[18][31:0]	00000012			Array
[17][31:0]	00000011			Array
[16][31:0]	00000010			Array
[15][31:0]	0000000f			Array
[14][31:0]	0000000e			Array
[13][31:0]	0000000d			Array
[12][31:0]	0000000c			Array
[11][31:0]	0000000b			Array
[10][31:0]	0000000a			Array
[9][31:0]	00000009			Array
[8][31:0]	00000008			Array
[7][31:0]	00000007			Array
[6][31:0]	00000006			Array
[5][31:0]	00000005			Array
[4][31:0]	00000004			Array
[3][31:0]	00000003			Array
[2][31:0]	00000002			Array
[1][31:0]	00000001			Array
[0][31:0]	00000000			Array

srav

(3) 对 54 条指令前仿真、后仿真、下板所用测试代码 coe 文件结果进行比对
使用所给的 mips_54_mars_simulate_student_ForWeb_2024.coe 导入 IP 核，使用给定的 testbench 文件 testbench_cpu54_multiple.v 进行前仿真，生成每条指令的 PC 值、指令值、寄存器值，并与测试平台比对结果文件 _246tb_ex10_result.txt 进行对比，结果完全一致，可以验证代码的正确性。



五、CPU 后仿真测试结果

(1) 对 CPU 后仿真的实验内容和结果进行描述，并给出 modelsim 后仿真波形图的贴图。

后仿真也称为时序仿真或者布局布线后仿真，是指电路已经映射到特定的工艺环境以后，综合考虑电路的路径延迟与门延迟的影响，验证电路能否在一定时序条件下满足设计构想的过程，是否存在时序违规。其输入文件为从布局布线结果中抽象出来的门级网表、Testbench 和扩展名为 SDO 或 SDF 的标准时延文件。SDO 或 SDF 的标准时延文件不仅包含门延迟，还包括实际布线延迟，能较好地反映芯片的实际工作情况。一般来说后仿真是必选的，检查设计时序与实际的 FPGA 运行情况是否一致，确保设计的可靠性和稳定性。选定了器件分配引脚后再做后仿真。

添加时钟约束

1.设置 create clock

Edit Create Clock

Creates a clock object. The created clock is applied to the specified source objects. If you do not specify source objects, but give a clock name, a virtual clock is created.

Clock name:

Source objects:

Waveform

Period: ns

Rise at: ns

Fall at: ns

☐ Add this clock to the existing clock (no overwriting)

Command: `create_clock -period 88.000 -name clk_pin -waveform {0.000 44.000} [get_ports clk_in]`

2.设置 Input Delay

Edit Set Input Delay

Specify input delay for ports or pins relative to a clock edge.

Clock:

Objects (ports):

Delay value: ns

Delay Value Options

Delay value is relative to clock edge:

Delay value already includes latencies of the specified clock:

Rise/Fall

☐ Delay value specifies delay

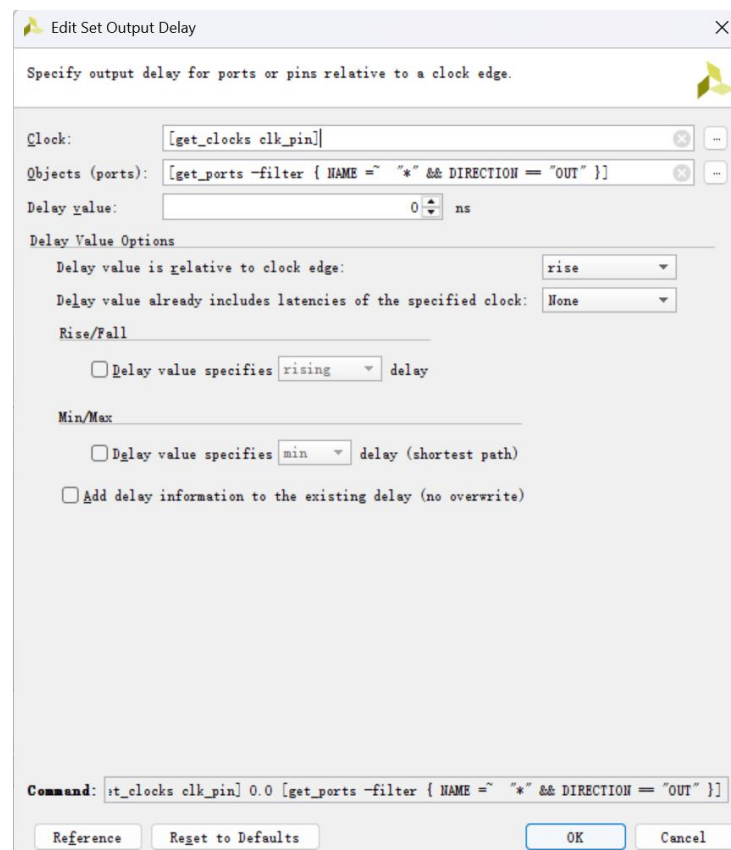
Min/Max

☐ Delay value specifies delay (shortest path)

☐ Add delay information to the existing delay (no overwrite)

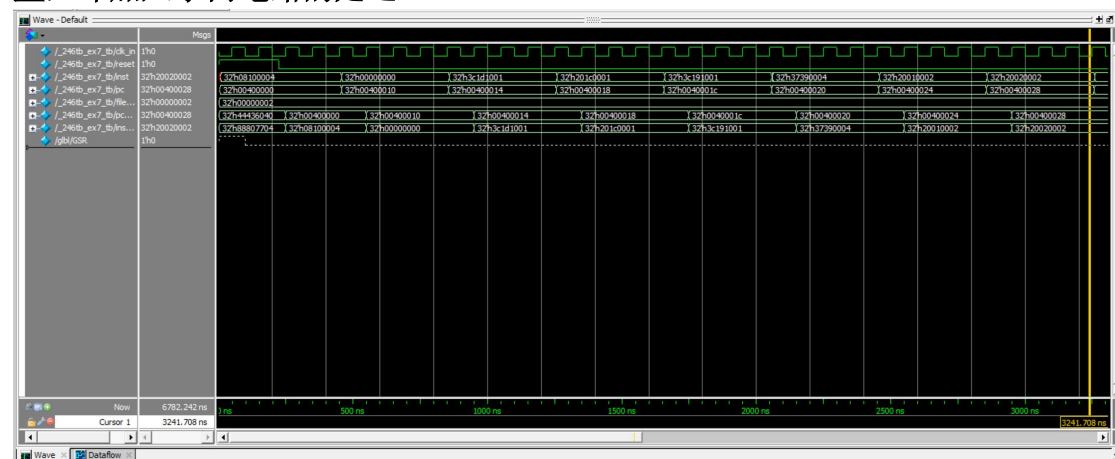
Command: `set_input_delay -clock [get_clocks clk_pin] 1.0 [get_ports reset]`

3.设置 Output Delay



后仿真 modelsim 波形图:

可以看到相比于前仿真后仿真上面每一个指令都相较于 PC 后移了一些位置, 即加入了门电路的延迟。



(2) 对时序分析的结果进行分析, 并截图。

设计时序摘要表(Design Timing Summary):

可以看到——"所有指定的时序约束均已满足", 这表明设计满足了所有时序要求, 没有违反时序约束的情况。

Design Timing Summary		
Setup	Hold	Pulse Width
Worst Negative Slack (VNS): 0.126 ns	Worst Hold Slack (VHS): 0.108 ns	Worst Pulse Width Slack (VPWS): 42.750 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 17997	Total Number of Endpoints: 17997	Total Number of Endpoints: 3592
All user specified timing constraints are met.		

Timing_2

Timing Summary - timing_3

Timing Summary - timing_4

Timing Summary - timing_5

Project Summary

sccomp_dataflow.v

datapath.v

ALU.v

testbench_cpu54_multiple.v

CPU54.xdc

D:/vivado_projects/CPU54/CPU54.srscs/constrs_1/new/CPU54.xdc

1

create_clock -period 88.000 -name clk_pin -waveform {0.000 44.000} [get_ports clk_in]

2

set_input_delay -clock [get_clocks clk_pin] 1.000 [get_ports reset]

3

set_output_delay -clock [get_clocks clk_pin] 0.000 [get_ports -filter { NAME == "s" && DIRECTION == "OUT" }]

4

得到的时序报告（timing_report）：

文件 编辑 查看

H1 列表 B I 撤销 重做

设置

Copyright 1986–2016 Xilinx, Inc. All Rights Reserved.

Tool Version : Vivado v.2016.2 (win64) Build 1577090 Thu Jun 2 16:32:40 MDT 2016

Date : Fri Jul 11 00:08:12 2025

Host : LAPTOP-VOAHCU9H running 64-bit major release (build 9200)

Command : report_timing_summary -delay_type min_max -report_unconstrained -check_timing_verbose -max_paths 10 -input_pins -name timing_5 -file D:/desktop/timing_report.txt

Design : sccomp_dataflow

Device : 7a100t-csg324

Speed File : -1 PRODUCTION 1.14 2014-09-11

Timing Summary Report

Timer Settings

Enable Multi Corner Analysis : Yes

Enable Pessimism Removal : Yes

Pessimism Removal Resolution : Nearest Common Node

Enable Input Delay Default Clock : No

Enable Preset / Clear Arcs : No

Disable Flight Delays : No

Ignore I/O Paths : No

Timing Early Launch at Borrowing Latches : false

Corner Name

Analyze Max Paths

Analyze Min Paths

Slow

Yes

Yes

Fast

Yes

Yes

check_timing report

Table of Contents

1. checking no_clock

2. checking constant_clock

3. checking pulse_width_clock

行 1, 列 1

1,365,020 个字符

纯文本

100%

Windows (CRLF)

UTF-8

六、CPU 下板结果

(1) 对 CPU 下板的实验内容和结果进行描述，并给出下板结果的贴图。

下板时,需要将 PC 寄存器或通用寄存器结果输出到七段数码管上。为了实现这个流程，我新建了一个顶层模块 top，一个分频器模块 Divider，和一个七段数码管显示模块 seg7x16。

FPGA 系统时钟为 100MHz，人眼无法观察指令执行过程。所以我使用分频器将时钟分频到 1Hz，方便人眼观察。

```
module Divider(  
    input clk,  
    input rst,  
    output reg clk_out  
);  
  
reg [31:0] counter = 0;  
always @(posedge clk or posedge rst) begin  
    if (rst) begin  
        counter <= 0;  
        clk_out <= 0;  
    end else begin  
        if (counter == 50000000) begin  
            clk_out <= ~clk_out;  
            counter <= 0;  
        end else begin  
            counter <= counter + 1;  
        end  
    end  
end  
endmodule
```

在顶层模块 top 中，我将 CPU 模块和分频器、七段数码管显示模块进行整合，100MHz 时钟驱动数码管的动态扫描，保证显示稳定不闪烁；1Hz 时钟控制指令执行的频率，也就是显示数据的更新频率，便于下板观察 CPU 指令执行的变化。

```
module top(  
    input  clk_in,          //时钟输入  
    input  reset,           //复位信号  
    output [7:0] o_seg,  
    output [7:0] o_sel  
);  
  
wire [31:0] inst;  
wire [31:0] pc;
```

```

wire clk_1hz;

Divider divider_inst(
    .clk(clk_in),
    .rst(reset),
    .clk_out(clk_1hz)
);

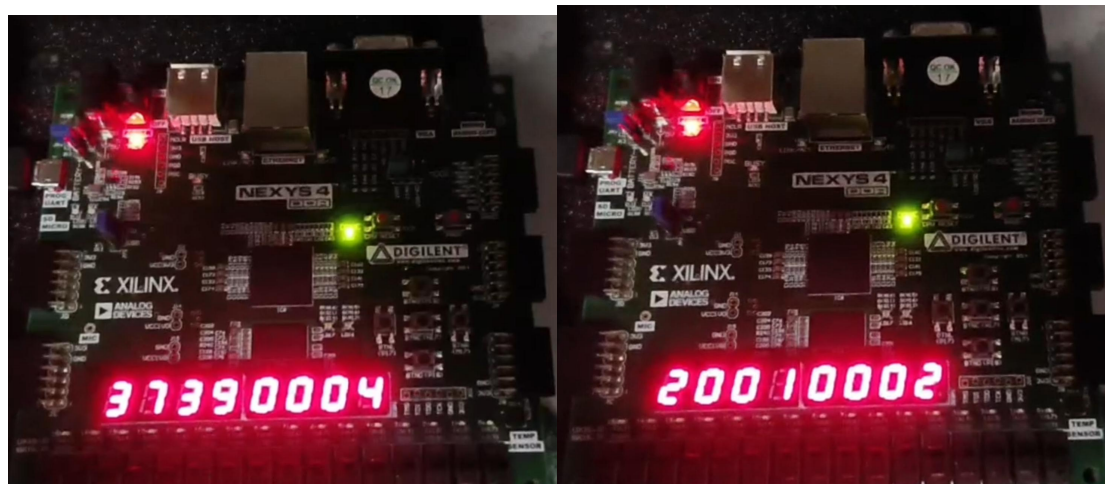
sccomp_dataflow sd(
    .clk_in(clk_1hz),
    .reset(reset),
    .inst(inst),
    .pc(pc)
);

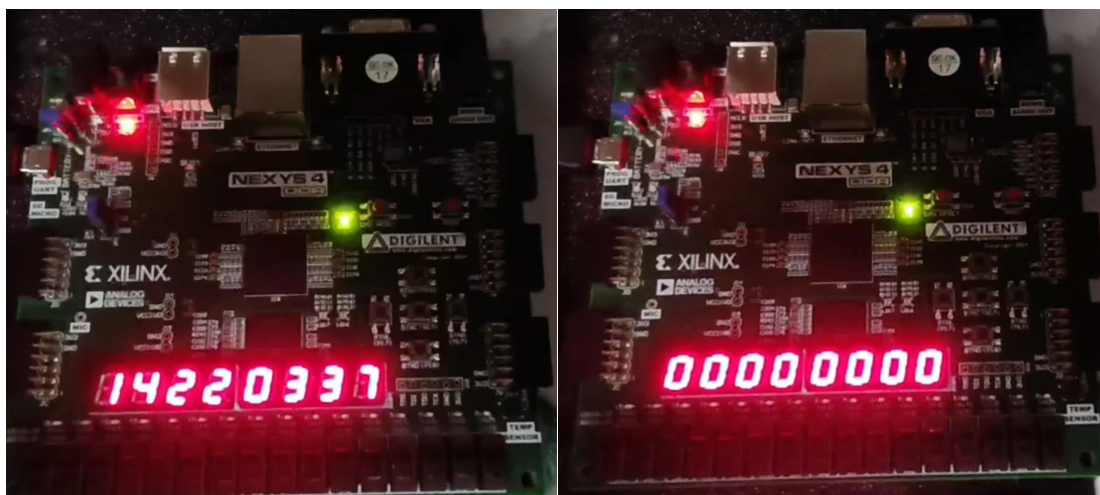
seg7x16 print(
    .clk(clk_in),
    .reset(reset),
    .cs(clk_1hz),
    .i_data(inst),
    .o_seg(o_seg),
    .o_sel(o_sel)
);

endmodule

```

下板结果：





(2) 实验的分析与结论。

在配置了 MIPS 网站提供的 board 下板 coe 文件后，下板后可以看到指令的执行过程，每秒钟代表一个时钟周期。由于不同指令所需的周期数不同，所以指令变化的时间间隔也不一样。

七、心得体会及建议

本次设计 54 条指令多周期 CPU 的实验让我受益匪浅，尤其是在设计的深度和复杂度上，相比之前的单周期 CPU 有了质的提升。

首先，本次实验让我完成了从单周期到多周期 CPU 设计的思维范式转换。单周期 CPU 的设计直观，但其性能完全受限于执行时间最长的那条指令，存在显著的“木桶效应”，导致整体时钟频率低下，资源利用率不高。多周期设计则将这一瓶颈彻底打破。通过将每条指令精心分解为取指（IF）、译码（ID）、执行（EX）、访存（MEM）、写回（WB）等更为细分的阶段，并让每个阶段在一个更短的时钟周期内完成，使得不同复杂度的指令可以占用不同的周期数，极大地提升了 CPU 的整体运行效率。这个过程让我深刻认识到，CPU 设计的核心之一便是对“时间”的精妙调度。而流水线寄存器（IR, ACC, MDR, Rs_Reg, Rt_Reg 等）正是实现这种调度的关键。它们作为阶段之间的“隔离带”和“数据渡船”，确保了在任何一个时刻，每个阶段处理的都是来自上一阶段的、稳定而正确的数据，从而避免了因 RegFile 等共享资源被后续指令占用而导致的数据冲突问题。

数据通路设计是本次实验的核心环节，其复杂程度远超单周期 CPU。本次设计不再是简单地将所有指令通路进行“或”运算式的叠加，而是在一个统一的、为多周期流水特殊设计的骨架上，进行需求驱动模块化扩展。为支持新增的 23 条指令，数据通路迎来了三个重要的结构性升级：第一，增加了独立的乘除法单元（MDU）以及专用的 HI/LO 寄存器，这打破了传统 R-Type 指令 Reg->ALU->Reg 的固定模式，形成了一条并行的、处理复杂运算的数据通路，并需要 mfhi/mflo 等指令作为与主寄存器堆交互的桥梁。第二，引入了 CP0 协处理器，它作为 CPU 的“特权管家”，内部集成了 EPC, Cause, Status 等核心状态寄存器，成为了实现整个异常和中断处理机制的硬件基石。第三，为了支持

lb/lh/sb/sh 等指令，设计了更精细的访存逻辑，包括为 DMEM 实现字节写使能（Byte Enables），以及在加载路径上增加加载数据处理器（LDP）来完成字节/半字的精确选择和符号/零扩展。这一系列设计让我体会到，一个完备的 CPU 数据通路是在核心通路上不断“挂载”专用功能模块的演进过程。

与数据通路相对应，控制部件（CU）的设计也从组合逻辑解码，演进为了一个复杂的有限状态机（FSM）。它成为了 CPU 真正的“大脑”。设计的核心是在数据通路的基础上列出一份详尽的“指令操作时间表”，这份表格将 54 条指令在每个状态下所需的所有控制信号值一一列出，成为了连接理论设计和 Verilog 代码实现的桥梁。在实现 FSM 时，我采用三段式写法，将状态寄存器的同步更新、次态的组合逻辑判断、以及输出控制信号的组合逻辑清晰地分离开来。这种结构不仅让代码的可读性和可维护性大大增强，也有效避免了在复杂逻辑中意外产生时序不稳的锁存器。其中，为处理 MDU 这种耗时不定的单元而设计的“启动-等待-完成”握手机制，以及为 syscall 等指令设计的强制状态跳转，都让我对控制器如何处理异步事件和特权操作有了深入的实践。

仿真测试是最后一步。相较于单周期，本次的测试维度也更为复杂。除了对每条指令的基本功能和边界数据进行“单元测试”外，重点在于对 CPU 的“特殊行为”进行验证。测试过程中，我在 ModelSim 波形图中仔细观察 CPU 是否能准确捕获异常、EPC 和 Cause 寄存器是否写入了正确的值、PC 是否强制跳转到了预设的异常处理入口，以及最终 eret 指令是否能让程序无缝地返回到中断点。其中，由于给出的 testbench 是 PC 值变化就打印一次，所以不能采用传统的在第一期就更新 PC 自增，否则会在同一条指令打印多次数据。为此我想到两种解决方法，第一种是设置一个 PC 寄存器，第二种就是在所有指令的最后一个周期再更新 PC 的值，也就是把 PC_in 信号放在最后一个周期拉高。为了代码修改的方便，本次我采用了第二种方法。但是这种方法应该会影响到后续的流水线 CPU 的设计，以后应该会再次修改。

总的来说，这次 54 条指令多周期 CPU 的设计，是一次从理论到实践的完整闭环。从指令集分析、数据通路搭建、控制单元实现到仿真验证和板级测试，更锻炼了我分析问题、解决问题以及动手实践的能力。在调试过程中，我和室友们不断测试、对比，互相帮助找出每一条指令的错误。定位一个微小的逻辑错误往往需要花费大量的时间和精力，但成功排除故障后的成就感也是切实的。它让我将计算机组成原理的知识体系融会贯通，系统性地掌握了从指令集分析、模块化数据通路构建、复杂 FSM 控制器实现，到多维度仿真验证的全流程。在面对和解决 jal 返回地址时序、IMEM 读取延迟等具体工程问题时，我的逻辑分析能力和动手调试能力得到了极大的锻炼。

这次实验不仅让我对流水线、中断、异步接口等现代处理器设计的核心思想有了更直观和深入的理解，更为我未来挑战更高级的五级流水线乃至更复杂的处理器设计，打下了坚实的基础，建立了强有力的信心。